

Energy Forecasting from Meteorological Data

LCP B Final Project by Brocco Luca, Cafagno
Samuele, Cavallin Jonathan, Di Lucia Sofia



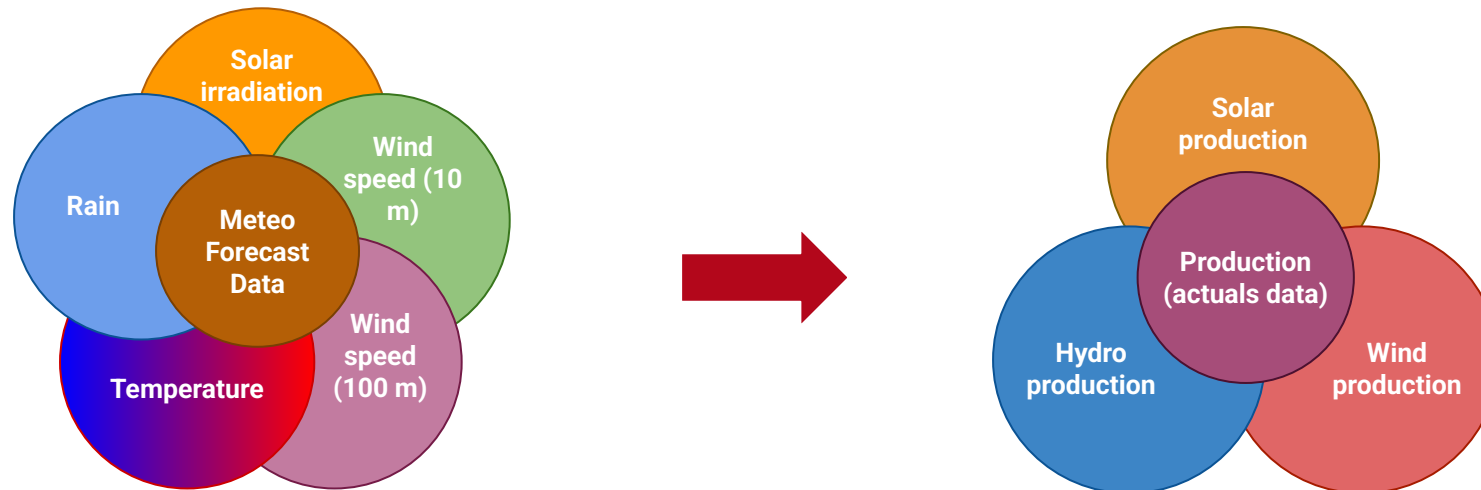
UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Project overview



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Our objective is to infer the renewable energy **production** starting from meteo **forecast** data



We assume meteo and production are **strongly** correlated

We aim to predict energy production over single time windows of **3 hours**



Several options for our datasets were available.

Our data covers:

- a **three years** period, going from 1/1/2022 to 31/12/2024, by timesteps of **3 hours**
- the whole of Italy, divided into market **zones**

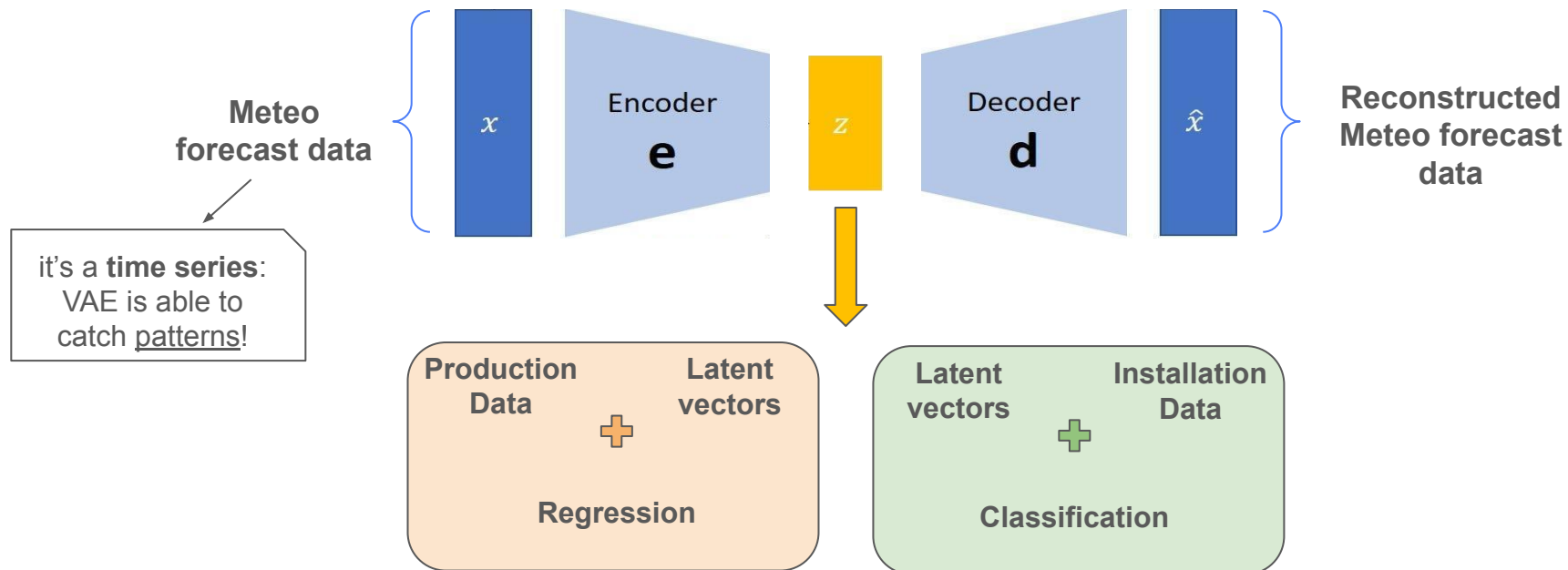
Italy's market zones are:

- IT-NORD
- IT-CNOR
- IT-CSUD
- IT-SUD
- IT-CALA
- IT-SICI
- IT-SARD





We will use a **Variational Auto Encoder (VAE)** for an encoding step, and a **Neural Network (NN)** for the regression / classification



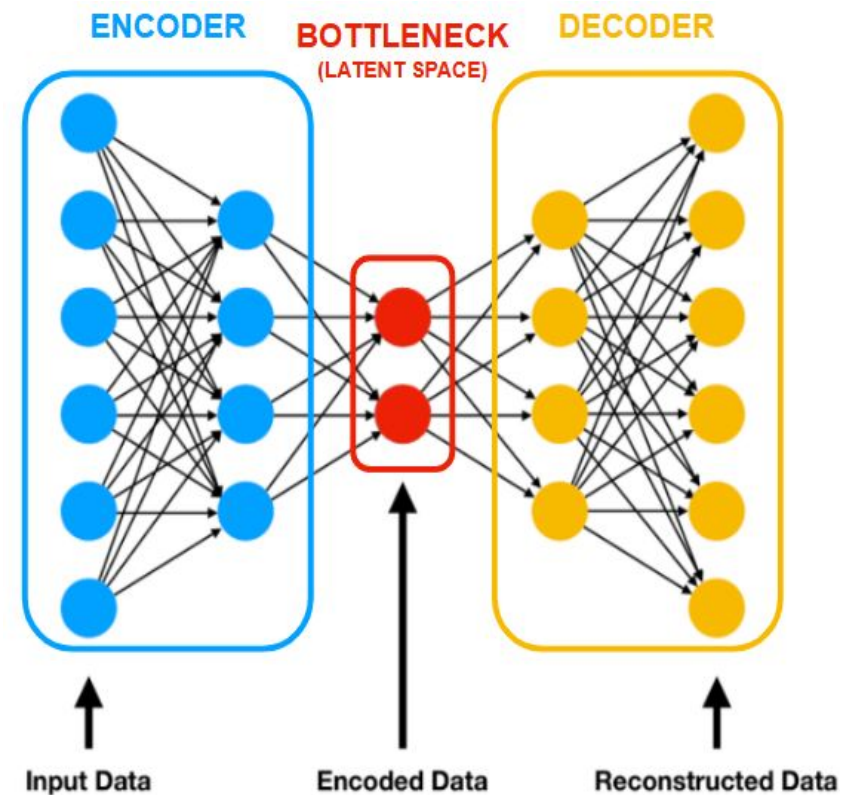
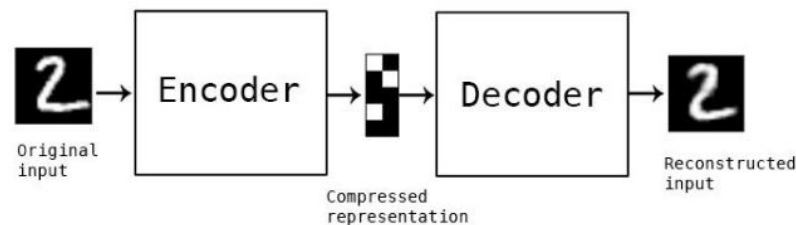
What is a Variational AutoEncoder?

6

Autoencoders (AEs) take data, compress it into a low-dimensional representation and then reconstruct it back into its original form.

Two components:

- **encoder**: takes the input data and compresses it
- **decoder**: attempts to reconstruct the input data



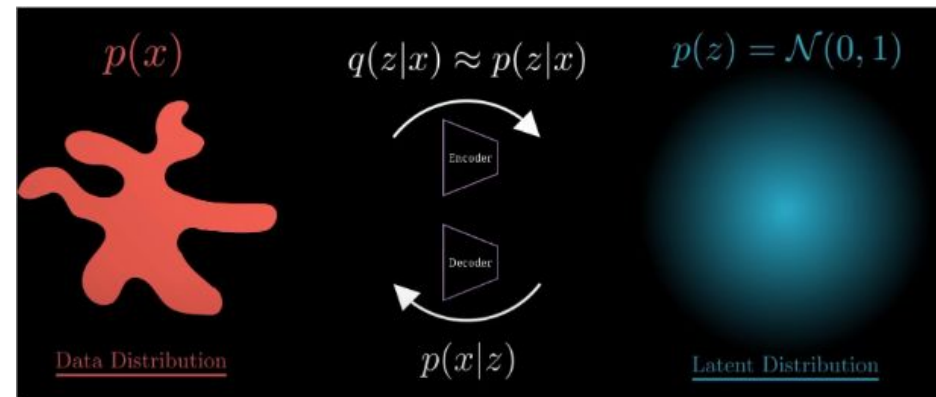
GOAL: to generate new data from a given distribution $p(x)$ which represents the dataset.
PROBLEM: we don't know the exact shape of $p(x) \rightarrow$ we introduce the **latent distribution $p(z)$** .

The two distributions live in different spaces $\rightarrow$ we introduce mappings to connect them:

1. (true) posterior distribution $p(z|x)$
2. likelihood distribution $p(x|z)$

ASSUMPTIONS:

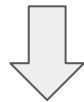
- the latent distribution is a **Gaussian distribution $p(z) = \mathcal{N}(0,1)$** .
- we approximate the posterior distribution with a **Gaussian distribution $p(z|x) \approx q(z|x) = \mathcal{N}(\mu, \sigma)$** .





Optimization of the **ELBO (Evidence Lower Bound)**, the training objective of the VAE:

$$\mathcal{L}(x^{(i)}, \theta, \phi) = \underbrace{\mathbb{E}_z \left[\log p_{\theta} \left(x^{(i)} \mid z \right) \right]}_{\text{Reconstruct the Input Data}} - \underbrace{D_{KL} \left(q_{\phi} \left(z \mid x^{(i)} \right) \parallel p_{\theta} (z) \right)}_{\text{KL Divergence}}$$

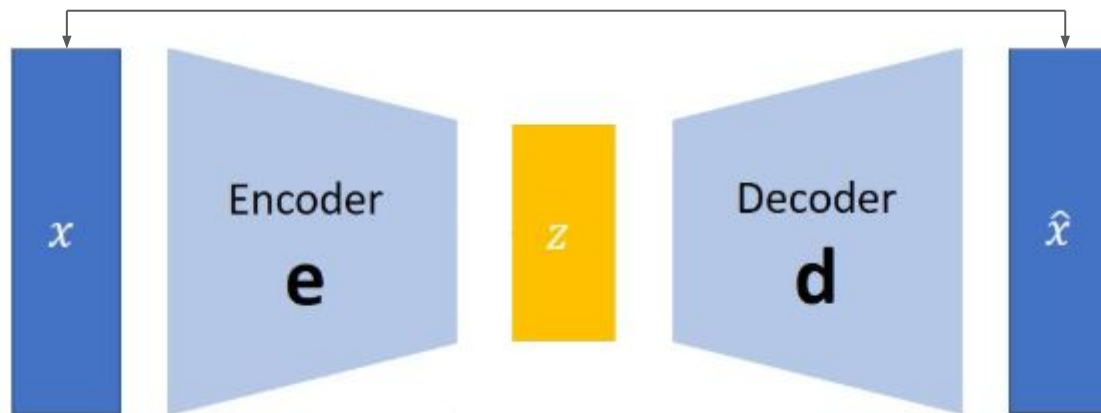


Data consistency (likelihood) term:
how well can the model reconstruct an image from its encoded version?
With the assumptions → **L2 reconstruction loss**



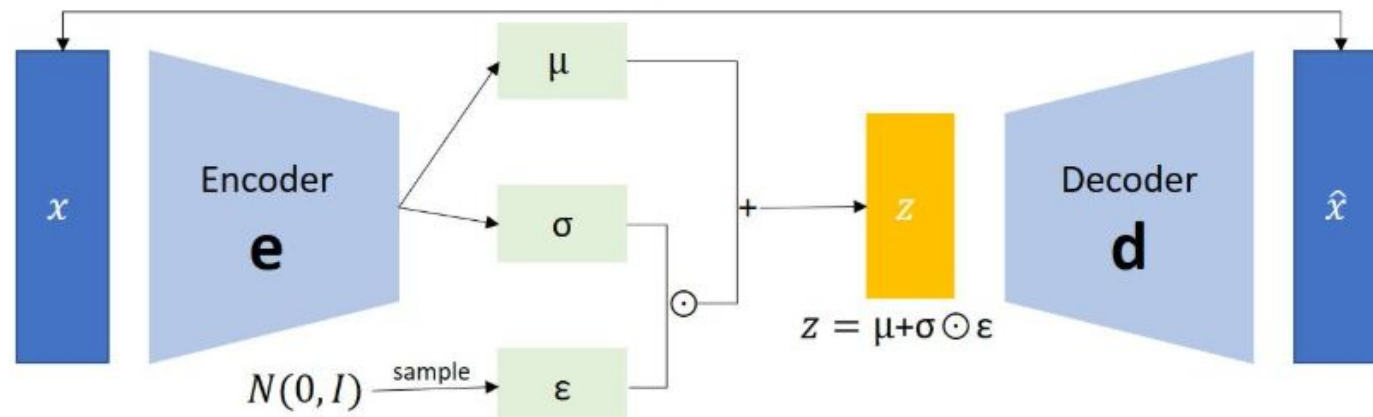
Regularization term:
‘distance’ between two probability distributions - how close $q(z|x)$ is to $p(z)$?
Impose a **normal distribution** on the latent space

Standard Autoencoder: input compressed by the encoder represented as a **single point** in the latent space.



1. The encoder compresses the input into a latent representation z
2. The decoder reconstructs the input
3. Training aims to minimize the Reconstruction Loss

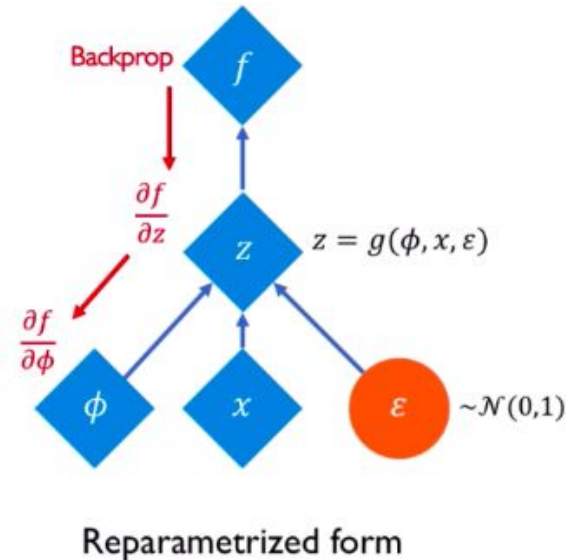
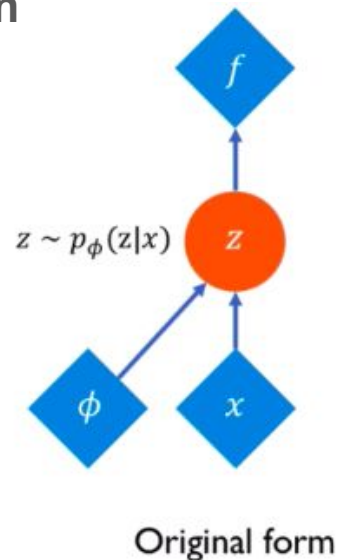
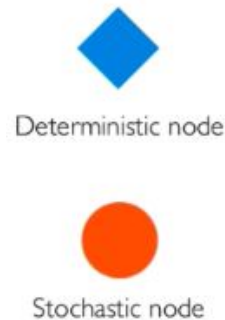
Variational Autoencoder: input compressed by the encoder represented as a **Gaussian distribution** in the latent space.



1. Sampling points from the latent distribution
2. The decoder converts these samples back into the input space
3. Computing the ELBO loss
4. **Backpropagation** ☐ **Reparameterization Trick**

1. We sample a random point ε from a Gaussian distribution
2. We scale it by the variance of the approximated posterior distribution: $\sigma \cdot \varepsilon$
3. We shift it by its mean: $\mu + \sigma \cdot \varepsilon$

⇒ the process becomes **differentiable** with respect to the mean μ and the variance σ
enabling backpropagation





We take an **input** data and pass it through the network



the network generates a **reconstructed version** of the input + the **parameters** (μ and σ)



we calculate the **ELBO loss**

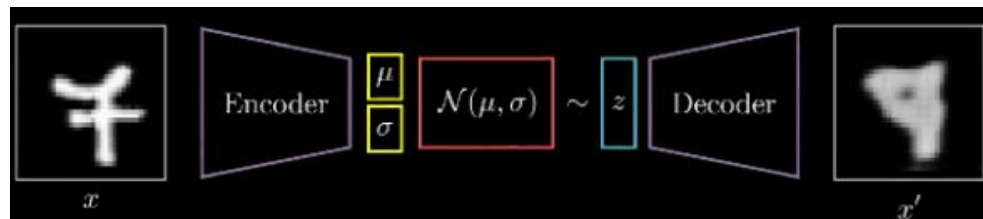


backpropagate this combined loss



update the network's **weights**.

We repeat this process for each input data.



Data preparation

Meteo Forecast data



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



We have 5 meteo forecast features:

- 2t (temperature)
- solar (solar irradiation)
- tp (rain precipitation)
- ws_10m (wind speed at 10 m)
- ws_100m (wind speed at 100 m)

Each Italy zone **has its own dataset**

Data preparation process, though, is the **same for all zones**

IT_SICI dataset

	timestamp	year	2t	solar	tp	ws_10m	ws_100m
0	2022-01-01 00:00:00	2022	280.463472	0.000000	1.493086e-07	2.103006	3.258443
1	2022-01-01 03:00:00	2022	282.004879	0.000000	1.493086e-07	2.157715	3.070916
2	2022-01-01 06:00:00	2022	281.652937	63444.125046	6.193800e-07	2.240972	3.290124
3	2022-01-01 09:00:00	2022	286.163287	319193.922752	6.193800e-07	2.295689	3.054874
4	2022-01-01 12:00:00	2022	288.954618	328800.995832	6.193800e-07	2.897822	3.033205
...	...	...	...	...	...	...	...
8025	2024-12-31 12:00:00	2024	286.402973	354148.954943	1.219816e-07	1.138562	1.008006
8026	2024-12-31 15:00:00	2024	284.948324	79227.011365	1.219816e-07	1.555412	1.850859
8027	2024-12-31 18:00:00	2024	280.286736	0.000000	1.990518e-07	1.466643	1.636367
8028	2024-12-31 18:00:00	2024	280.286736	0.000000	1.990518e-07	1.466643	1.636367
8029	2024-12-31 21:00:00	2024	279.211469	0.000000	1.990518e-07	1.154217	1.532170

8030 rows × 15 columns



Solar and **rain** data are acquired only every 12 hours, but we want every 3 hours

We can estimate the solar irradiation at given times with **Lambert's law**:

$$I(t) = I_0 \cdot \cos(\theta_z(t))$$

with $\theta_z(t)$ being the zenithal angle defined as $\theta_z(t) = 90^\circ - \alpha(\psi, \delta)$, with

α : altitude, ψ : latitude, δ : solar declination

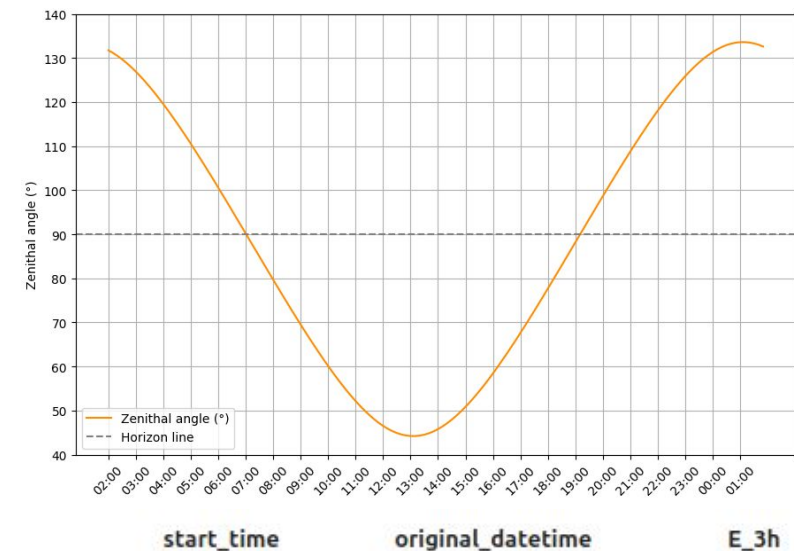
Both altitude and solar declination depend on the date and time of the day. We used the **average coordinates** of the **2 biggest cities** of each zone as coordinates

We used a specialized library for simulating the performance of photovoltaic systems to perform such calculations (**pvl**)



- We estimated I_0 as a normalization factor of the cosine function for each of the 6 hours period.
- Interpolated the points obtained calculating analytically $I(t)$ every hour
- Integrated the interpolated curve over 3 hours time periods

The night periods are managed by only considering positive zenithal angles (when the sun is above the horizon line) and setting irradiation to zero otherwise.



2022-01-01 03:00:00	2021-12-31 18:00:00+00:00	0.000000
2022-01-01 06:00:00	2022-01-01 06:00:00+00:00	16900.948502
2022-01-01 09:00:00	2022-01-01 06:00:00+00:00	206381.447532
2022-01-01 12:00:00	2022-01-01 06:00:00+00:00	240626.243224
2022-01-01 15:00:00	2022-01-01 06:00:00+00:00	52969.288800



Rain precipitations **do not have a clear time schema**: the easiest way to recreate data is to assume a **uniform distribution** across the single time steps and obtain data of 3h timesteps.

This way we are ignoring the true time distribution of rain, though it could still be informative in terms of meteo forecast (cloudy days, foggy, clear, etc...).

start_time	original_datetime	tp
2022-01-01 06:00:00+00:00	2022-01-01 06:00:00+00:00	2.432269e-07
2022-01-01 09:00:00+00:00	2022-01-01 06:00:00+00:00	2.432269e-07
2022-01-01 12:00:00+00:00	2022-01-01 06:00:00+00:00	2.432269e-07
2022-01-01 15:00:00+00:00	2022-01-01 06:00:00+00:00	2.432269e-07
2022-01-01 18:00:00+00:00	2022-01-01 18:00:00+00:00	4.707134e-07
2022-01-01 21:00:00+00:00	2022-01-01 18:00:00+00:00	4.707134e-07
2022-01-02 00:00:00+00:00	2022-01-01 18:00:00+00:00	4.707134e-07
2022-01-02 03:00:00+00:00	2022-01-01 18:00:00+00:00	4.707134e-07



Time related features have to be handled correctly to preserve **cyclicity**

VAE does not know what a date such as DD/MM/YYYY means, so we take the day number (from 1 to 365/366) and compute two **new time variables**:

$$x_{t1} = \sin(2\pi \text{ \#day} / \text{total year days})$$

$$x_{t2} = \cos(2\pi \text{ \#day} / \text{total year days})$$

This allows for easy handling of cyclicity. We also consider information on day, month, year and hour separately.



We know that meteo forecast is **strongly correlated** with past weather, so we adopt a **sliding windows** approach: each row of input data contains also the **8 previous meteorological and temporal records** (each record considers the mean value in a 3h timestep).

This way we suggest to VAE that each record is correlated with data from the last **24h**

	timestamp	year	2t	solar	tp	ws_10m	ws_100m	hour_sin	hour_cos	day_sin	day_cos	month_sin	month_cos	sin_dayofyear	cos_dayofyear
0	2022-01-01 00:00:00	2022	280.463472	0.000000	1.493086e-07	2.103006	3.258443	0.000000e+00	1.000000e+00	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
1	2022-01-01 03:00:00	2022	282.004879	0.000000	1.493086e-07	2.157715	3.070916	7.071068e-01	7.071068e-01	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
2	2022-01-01 06:00:00	2022	281.652937	63444.125046	6.193800e-07	2.240972	3.290124	1.000000e+00	6.123234e-17	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
3	2022-01-01 09:00:00	2022	286.163287	319193.922752	6.193800e-07	2.295689	3.054874	7.071068e-01	-7.071068e-01	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
4	2022-01-01 12:00:00	2022	288.954618	328800.995832	6.193800e-07	2.897822	3.033205	1.224647e-16	-1.000000e+00	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
5	2022-01-01 15:00:00	2022	287.767984	73977.377423	6.193800e-07	2.701854	3.762610	-7.071068e-01	-7.071068e-01	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
6	2022-01-01 18:00:00	2022	285.070379	0.000000	3.428422e-07	2.696382	4.195301	-1.000000e+00	-1.836970e-16	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
7	2022-01-01 21:00:00	2022	285.070379	0.000000	3.428422e-07	2.696382	4.195301	-1.000000e+00	-1.836970e-16	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
8	2022-01-01 00:00:00	2022	282.471923	0.000000	3.428422e-07	2.518416	3.825331	-7.071068e-01	7.071068e-01	0.201299	0.979530	0.5	0.866025	0.017213	0.999852
9	2022-01-02 00:00:00	2022	281.653937	0.000000	3.428422e-07	2.323606	3.379215	0.000000e+00	1.000000e+00	0.394356	0.918958	0.5	0.866025	0.034422	0.999407

duplicated

RESULT:

t_{-8}	t_{-7}	...	t_{-1}	t_0
$t+1_{-8}$	$t+1_{-7}$	...	$t+1_{-1}$	$t+1_0$
$t+2_{-8}$	$t+2_{-7}$	...	$t+2_{-1}$	$t+2_0$
...	...	...	...	...

original

Data preparation

Production and Installation data



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



We have 2 datasets:

- **actuals** contains **effective production** and load of the network
- GSE¹ data related to **installations** for all renewable energy sources

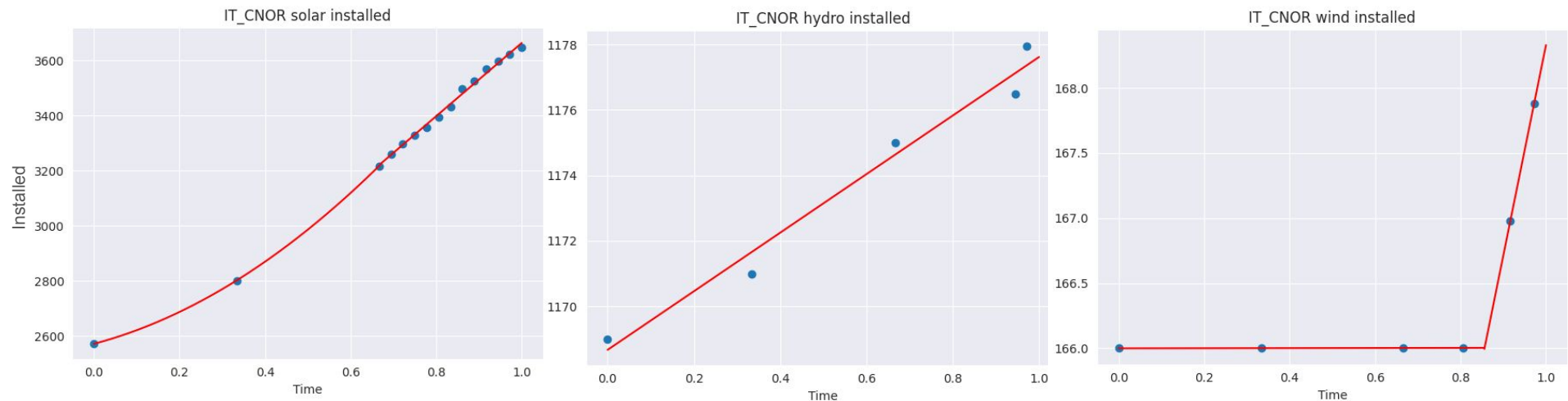
We want to study **weather - production correlation**: actuals dataset suffers a **trend** given by growing installed power.

We tackle the problem in two steps:

- 1) **Fit power installation data** (data is available only for limited timestamps) and give an estimate of installation for each source at every time t
- 2) **Detrend** the production by **normalizing** to the installation power at a given time.

We perform these operations separately on all zones.

1. <https://www.gse.it/dati-e-scenari/statistiche>



Few points were available from 2021 to 2023: we (mostly) fitted solar according to a parabolic trend, while others with linear trends

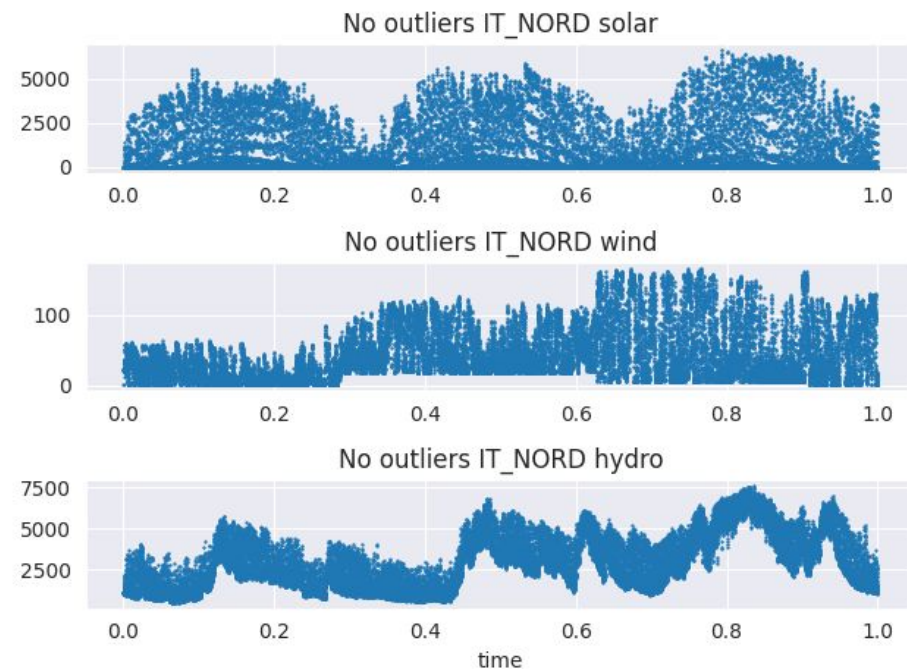
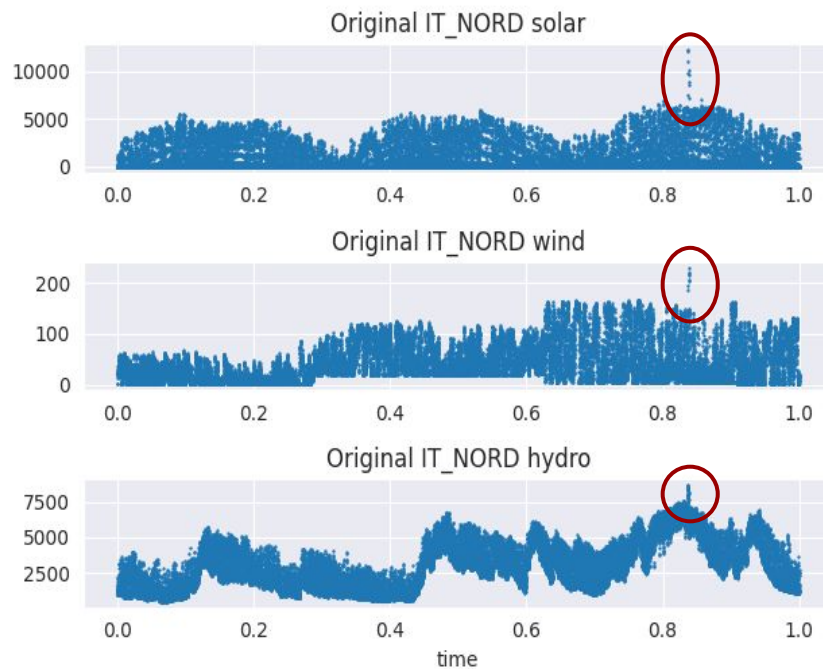
Fits strongly change between zones (installations are localized)

	solar	wind	hydro	generation	load	time	solar_installed	hydro_installed	wind_installed
date_time									
2022-01-01 00:00:00+00:00	0.0	19.0	176.0	1438.0	1821.0	0.000000	2571.170000	1168.667728	165.999198
2022-01-01 01:00:00+00:00	0.0	25.0	153.0	1386.0	1691.0	0.000038	2571.185645	1168.668069	165.999198
...	...	...	...	...	...	...	...	...	...
2024-12-30 19:00:00+00:00	0.0	4.0	164.0	1726.0	2839.0	0.999848	3663.933202	1177.619517	168.327534
2024-12-30 20:00:00+00:00	0.0	2.0	151.0	1733.0	2655.0	0.999886	3663.983845	1177.619858	168.328151
2024-12-30 21:00:00+00:00	0.0	3.0	111.0	1664.0	2371.0	0.999924	3664.034488	1177.620199	168.328767
2024-12-30 22:00:00+00:00	0.0	4.0	93.0	1649.0	2121.0	0.999962	3664.085131	1177.620539	168.329384
2024-12-30 23:00:00+00:00	0.0	3.0	68.0	1575.0	1916.0	1.000000	3664.135774	1177.620880	168.330000

26280 rows × 9 columns

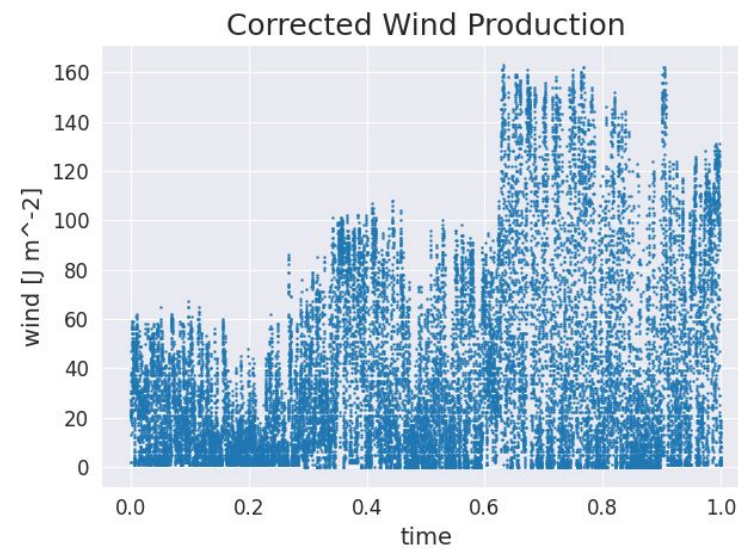
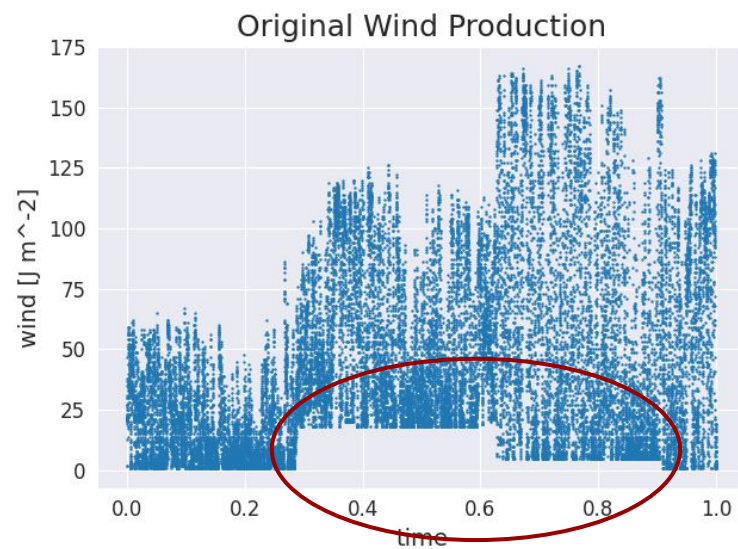


We noticed some weird **outliers** in actuals dataset. We **removed** them





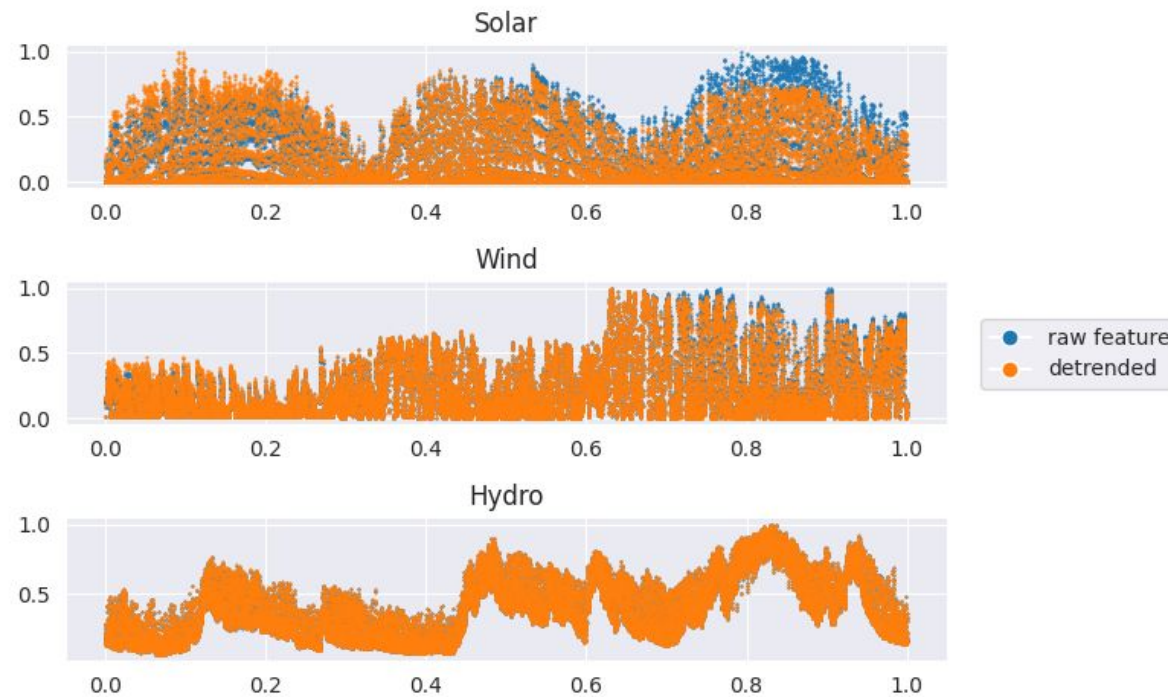
Some features in actuals presented unjustified **offsets**. We subtracted the offset to the original data to have a more plausible trend (IT_NORD data):





Finally, we **normalized** the actuals production data by the fitted estimate of installation at each timestep. The effect is most visible for the **Solar** feature.

IT_NORD data:





Our final dataset contains **detrended features** that will be used for regression and **installation features**, as well as **ratio**, useful for classification

$$ratio = \frac{solar + wind + hydro}{load}$$

date_time	solar	wind	hydro	generation	load	time	solar_installed	hydro_installed	wind_installed	detrended_solar	detrended_wind	detrended_hydro	ratio
2022-01-01 00:00:00+00:00	0.0	2.0	1110.0	6894.0	10583.0	0.000000	10196.340000	10149.666667	164.000000	0.0	0.012195	0.109363	0.105074
2022-01-01 01:00:00+00:00	0.0	22.0	1063.0	6297.0	10054.0	0.000038	10196.391682	10149.670776	164.003653	0.0	0.134143	0.104732	0.107917
2022-01-01 02:00:00+00:00	0.0	22.0	1057.0	5373.0	9573.0	0.000076	10196.443387	10149.674886	164.007306	0.0	0.134140	0.104141	0.112713
2022-01-01 03:00:00+00:00	0.0	22.0	1089.0	4963.0	9402.0	0.000114	10196.495115	10149.678996	164.010959	0.0	0.134137	0.107294	0.118166
2022-01-01 04:00:00+00:00	0.0	22.0	1053.0	5405.0	9689.0	0.000152	10196.546866	10149.683106	164.014612	0.0	0.134134	0.103747	0.110951
...	...	...	...	...	...	...	...	...	...	...	...	...	...
2024-12-30 19:00:00+00:00	0.0	14.0	2271.0	15045.0	16115.0	0.999848	17204.457726	10243.570173	208.270793	0.0	0.067220	0.221700	0.141793
2024-12-30 20:00:00+00:00	0.0	8.0	1609.0	13161.0	14848.0	0.999886	17204.737136	10243.573115	208.270795	0.0	0.038412	0.157074	0.108904
2024-12-30 21:00:00+00:00	0.0	8.0	1313.0	11398.0	13473.0	0.999924	17205.016547	10243.576057	208.270797	0.0	0.038412	0.128178	0.098048
2024-12-30 22:00:00+00:00	0.0	8.0	1201.0	10413.0	12264.0	0.999962	17205.295958	10243.578998	208.270798	0.0	0.038412	0.117244	0.098581
2024-12-30 23:00:00+00:00	0.0	3.0	1379.0	9291.0	11071.0	1.000000	17205.575369	10243.581940	208.270800	0.0	0.014404	0.134621	0.124831

26239 rows × 13 columns

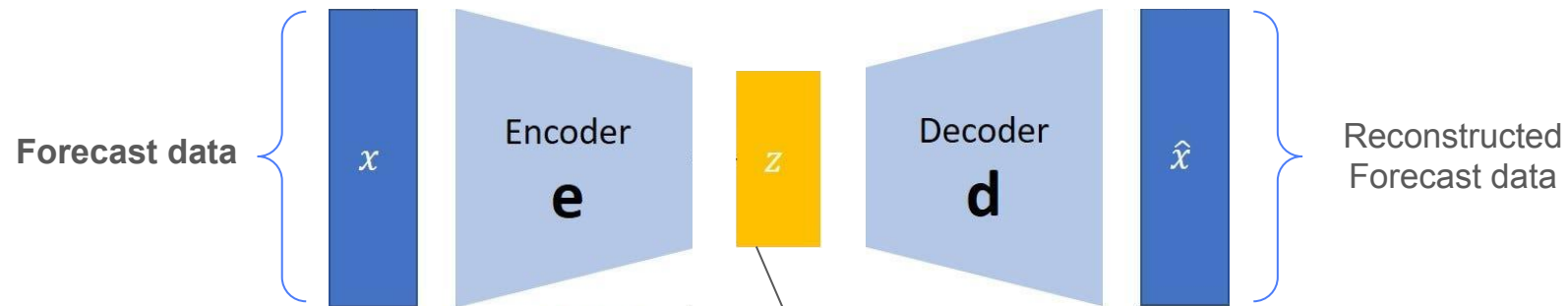
Classification

VAE and NN separated

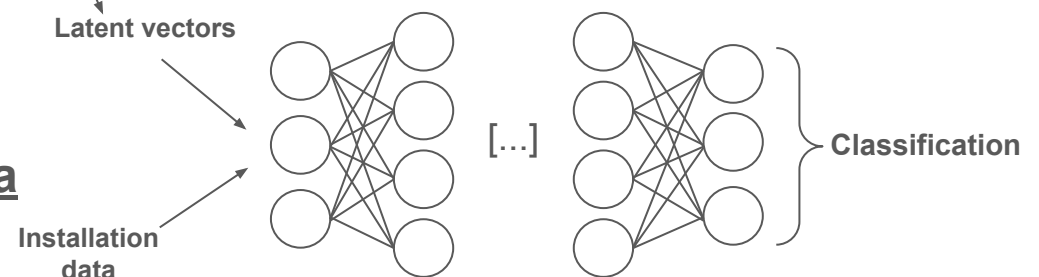


UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Our objective is to use a VAE to **reduce the dimensionality** of the dataset
VAE can capture the most informative and relevant features out of all inputs



We then take the latent space vectors and use them as input for a NN
NN is also feeded with installation data
NN output is the desired **classification**



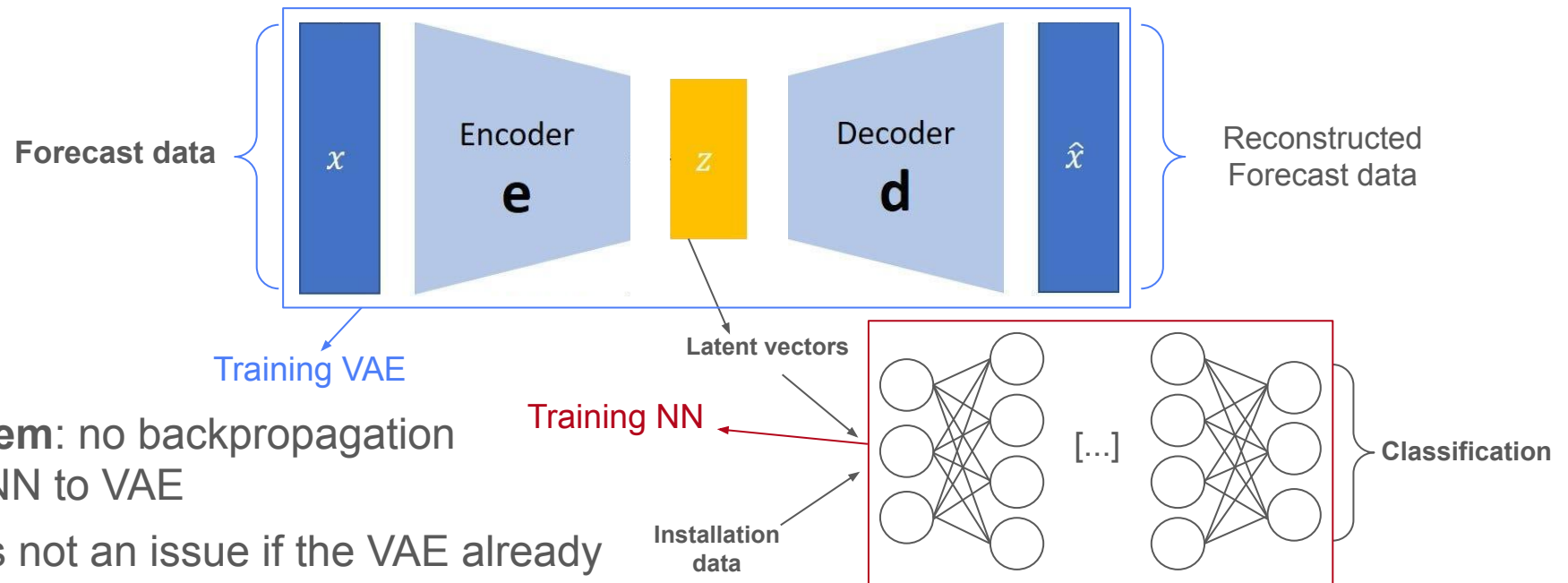


Classification (VAE and NN split)



The training is performed **separately** for the VAE and the NN!

The main assumption is having **inherent strong correlations** in Forecast data



Problem: no backpropagation
from NN to VAE

This is not an issue if the VAE already
produces a **well-clustered space**



Wouldn't a **PCA** achieve the same outcome?

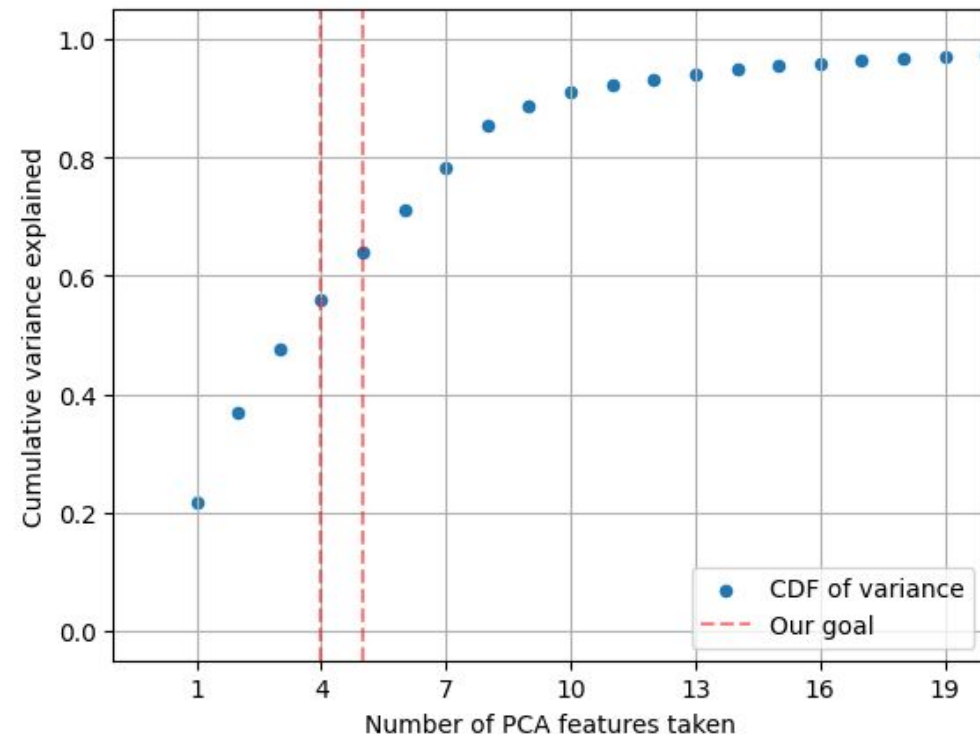
We tried:

```
shape of input: (8320, 90)
```

8320 rows x 90 features

if we set our goal to keep only 4-5 features, we have to give up ~40% of the whole dataset's variance!

PCA is not reliable for this task!

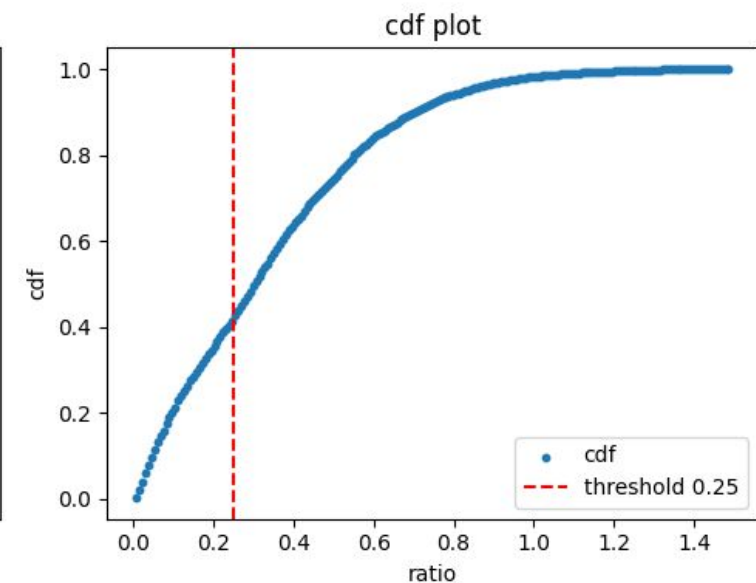
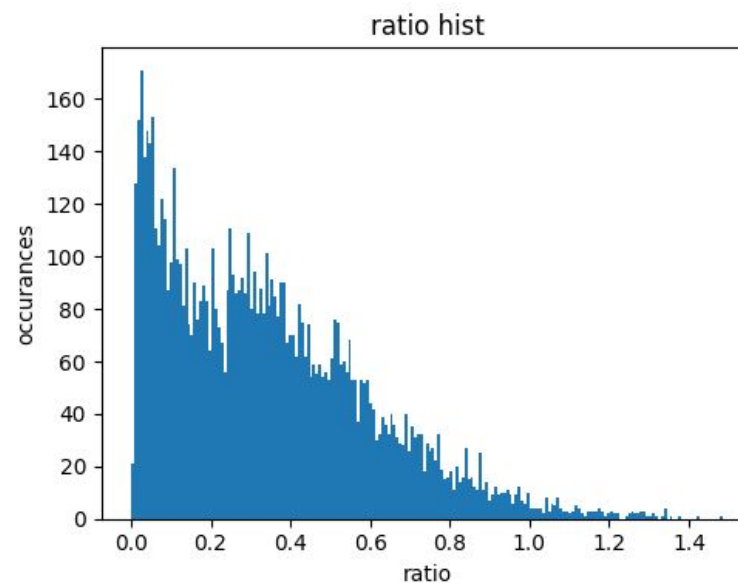




Problem: production is **far too low** to satisfy the load (IT_SICI data):

We set a custom **ratio threshold** for each zone to divide in equally distributed classes

ZONE	thresh
NORD	0.25
CNOR	0.2
CSUD	0.25
SUD	0.75
CALA	0.75
SICI	0.25
SARD	0.5

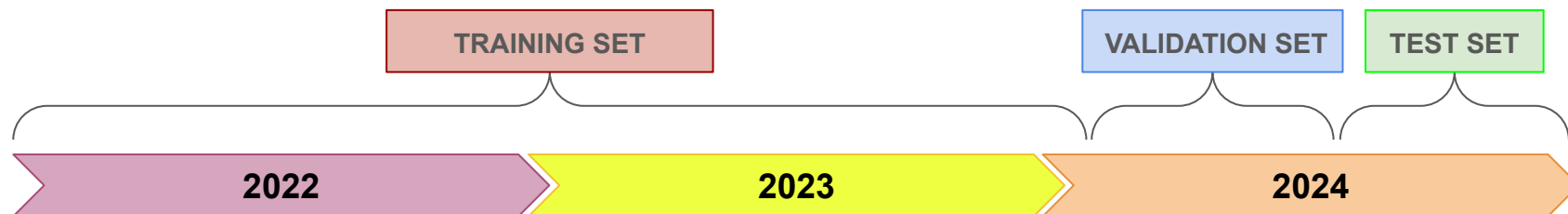




Since we are training on a time series, our goal is to use **past data** to predict the short term (and possibly long term) **future**.

The dataset split has to be done accordingly: the first part of the dataset (**70%** of datapoints) is used as **training** set, the following **15%** as **validation** set and the rest as **test** set.

This setting is worse than others in terms of accuracy, but ensures the model is **meaningful** with respect to our goal.





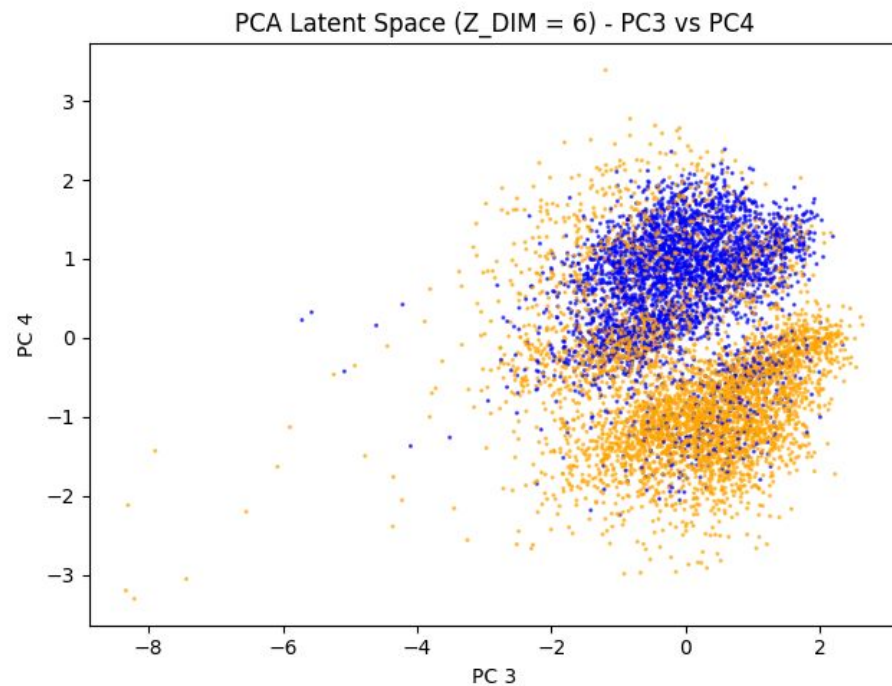
We trained only **1 layer** of Encoder and Decoder (symmetric)
If needed we can adjust the structure of the VAE

```
# parameters for grid search
param_grid = {
    'H_DIM': [16, 32, 48, 64],
    'Z_DIM': [4, 5, 6],
    'LR_RATE': [1e-3, 1e-4, 1e-5],
    'BATCH_SIZE': [128, 64],
    'NUM_EPOCHS': [20]
}
```

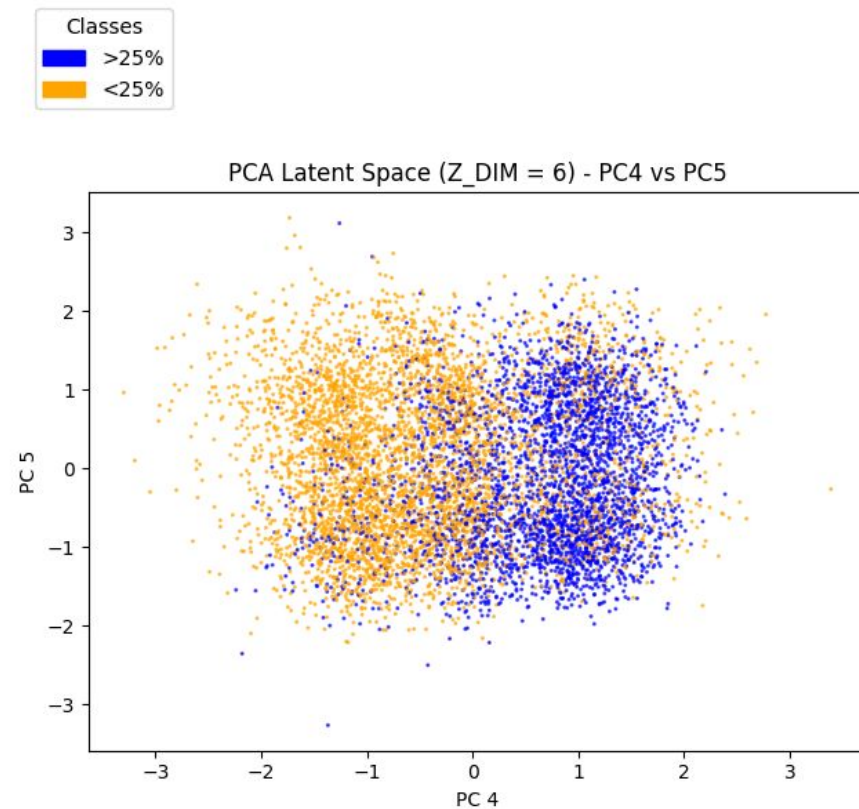
We implemented a **grid search** on the hyperparameters: the best model is then trained on more epochs.

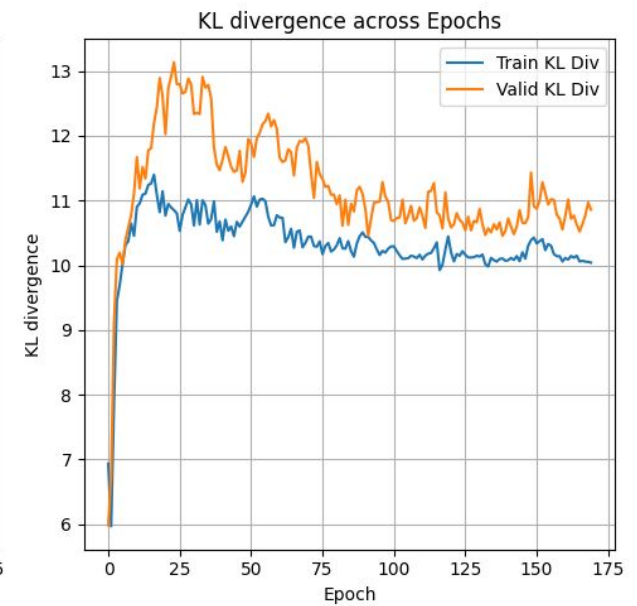
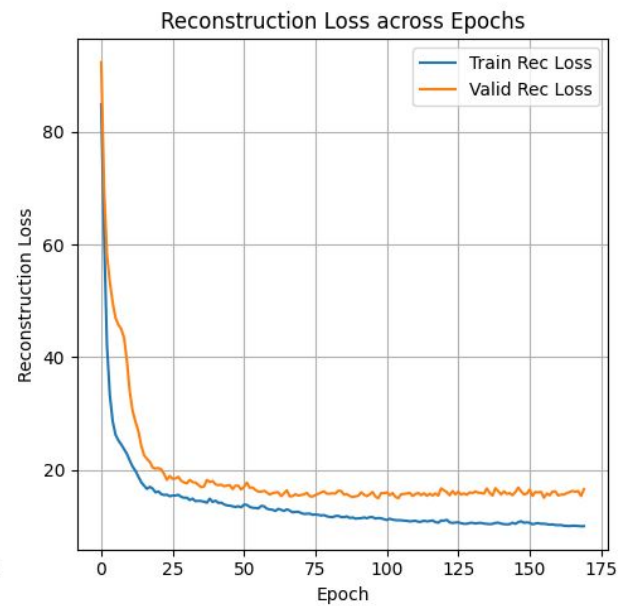
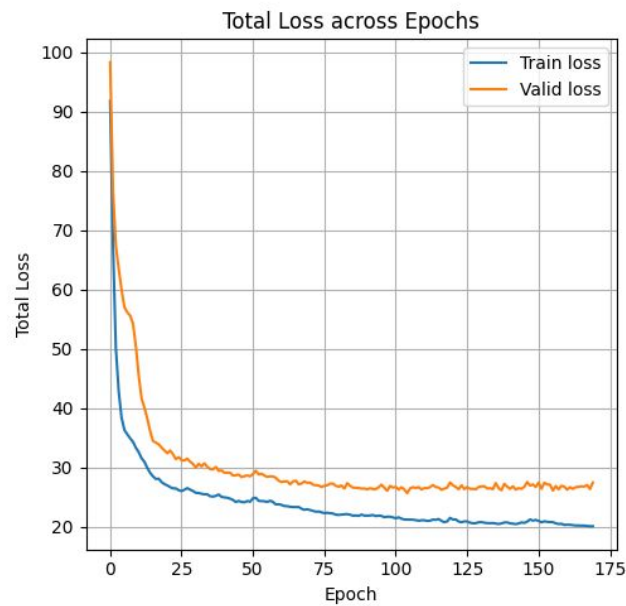
Best loss: 34.55871928540653

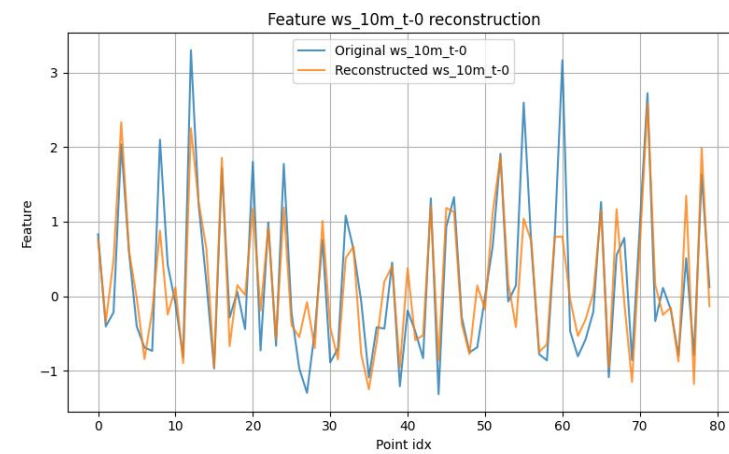
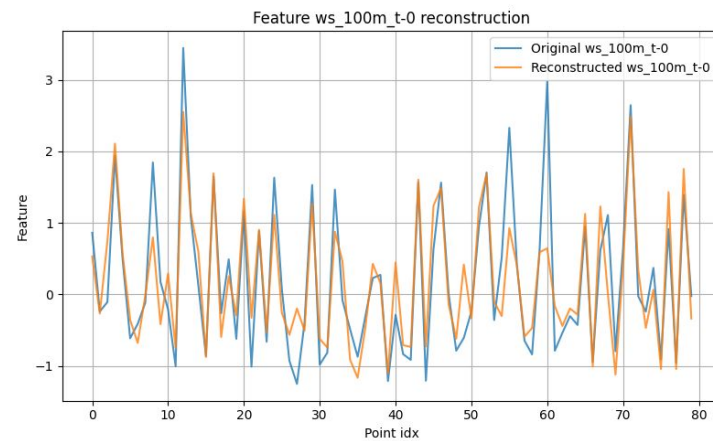
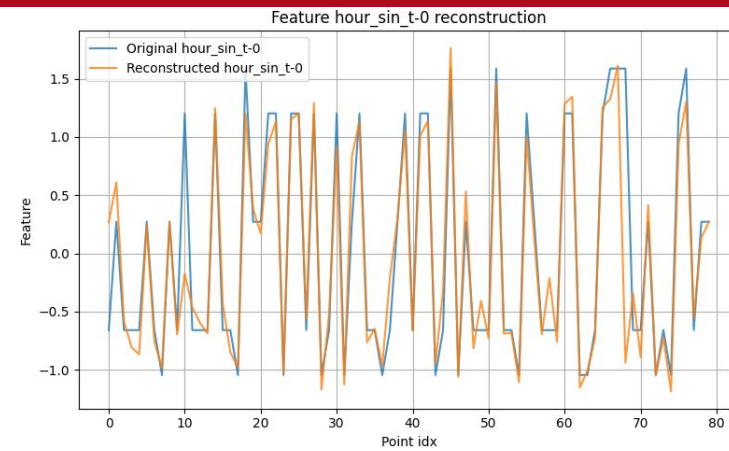
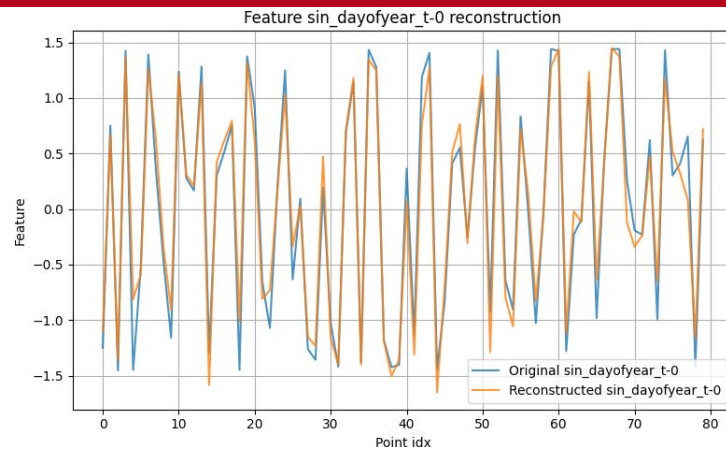
Best Params: {'H_DIM': 64, 'Z_DIM': 6, 'LR_RATE': 0.001, 'BATCH_SIZE': 64, 'NUM_EPOCHS': 20}

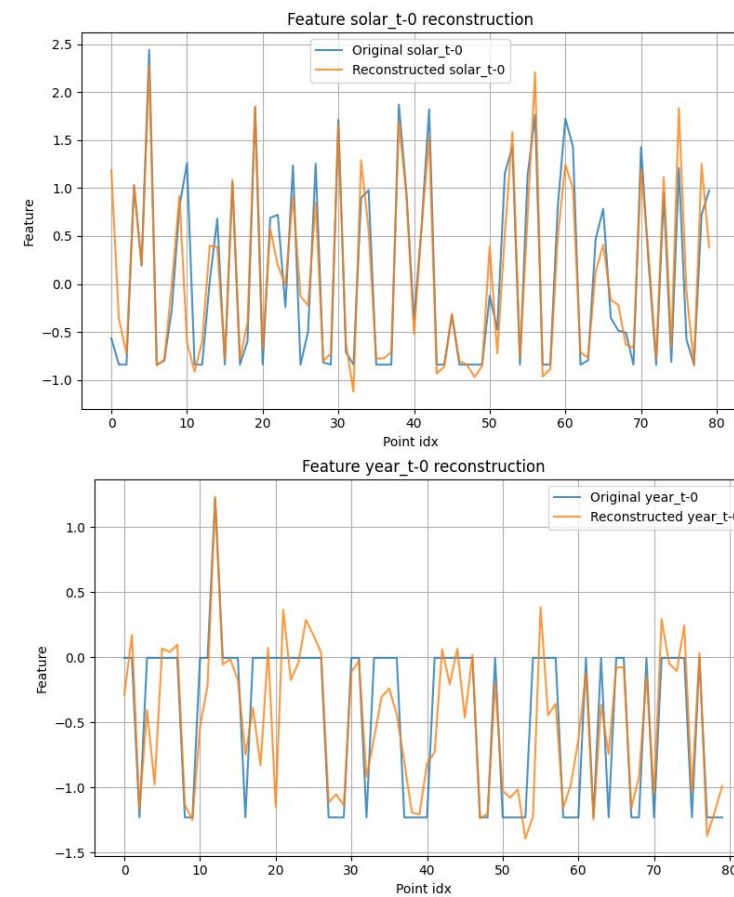
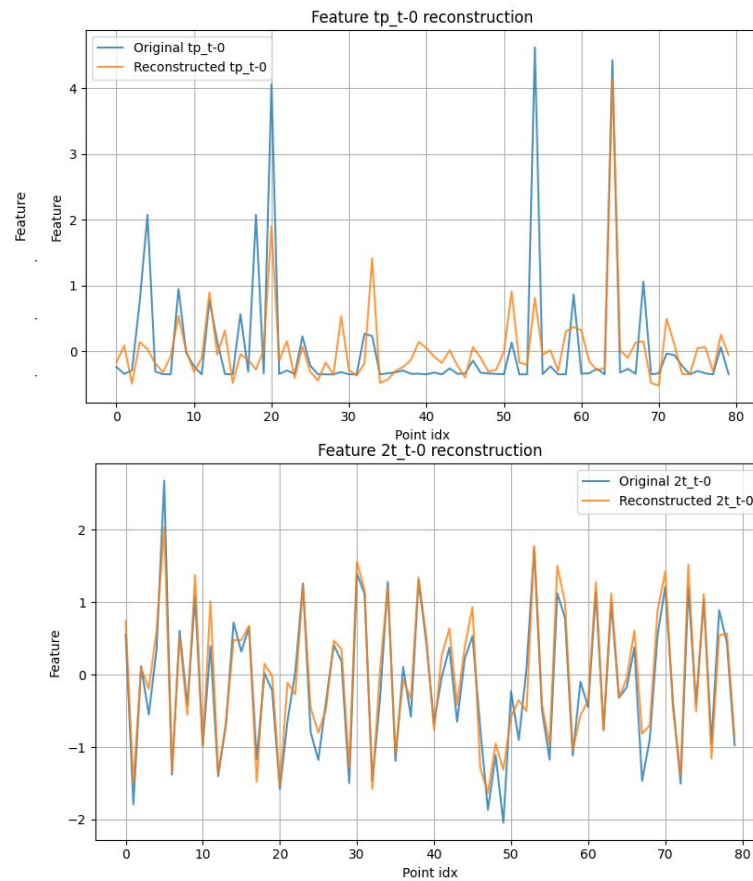


Latent space looks nice: we can even try **clustering** on it!











VAE reconstructions and latent space seem promising: let's train the NN!

```
All different classes: tensor([0, 1])
zs_tensor shape: torch.Size([8091, 6])
X_extra shape: torch.Size([8091, 3])
```

```
# Define the search space
param_grid = {
    'hidden_dims': [[64, 32], [32, 16]],
    'lr': [1e-4, 1e-3],
    'batch_size': [64, 128],
    'weight_decay': [1e-2, 1e-3, 1e-4],
    'dropout_prob': [0.05, 0.15, 0.3]
}
```

Best Hyperparameters:

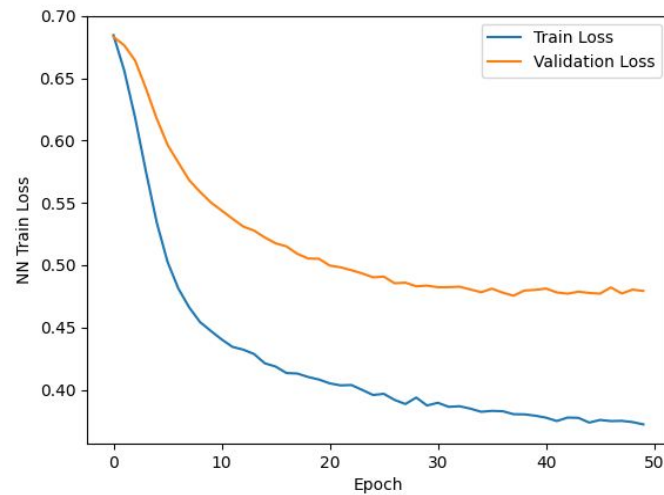
```
{'hidden_dims': [32, 16], 'lr': 0.0001, 'batch_size': 128, 'weight_decay': 0.001, 'dropout_prob': 0.15}
```

Best Validation Accuracy: 91.99%

NN inputs have variable dimensions: **4-6** from latent space and **3** extra inputs from actuals (3 installation features)

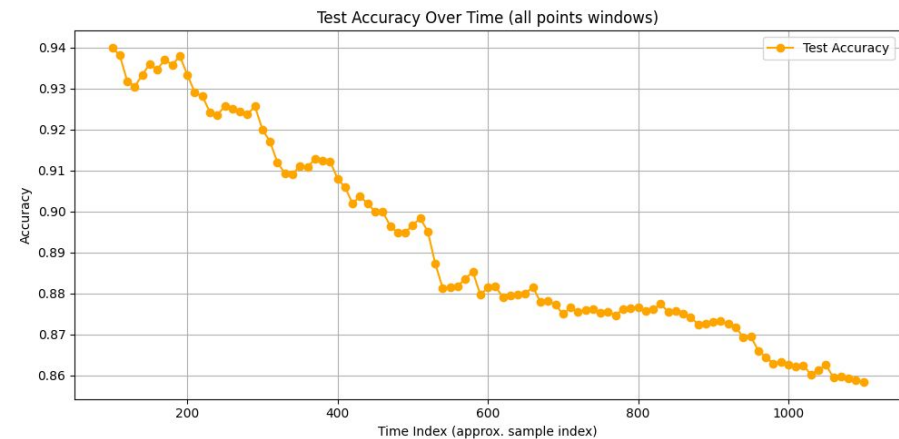
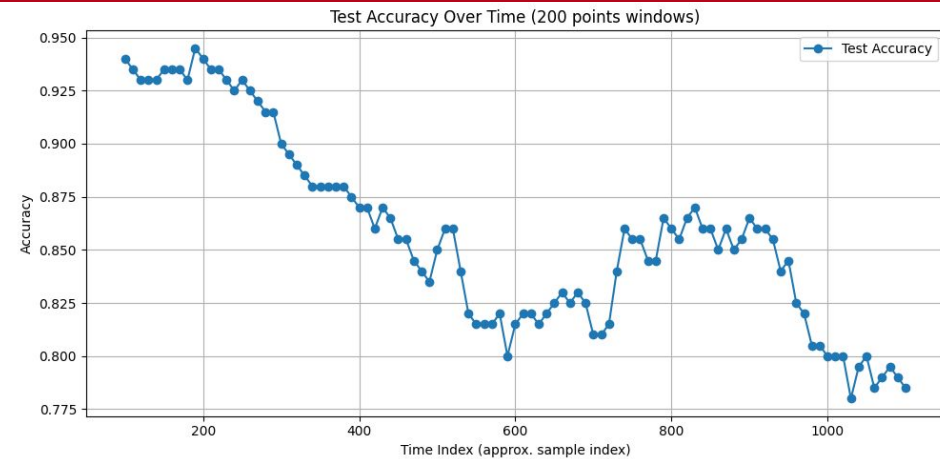
We run a simple grid search to seek for best params

We implement both weight decay and dropout to **prevent overfitting**



Other than loss trend, we checked the **accuracy over time** for all zones

We expect a decreasing trend, with some occasional peaks or drops

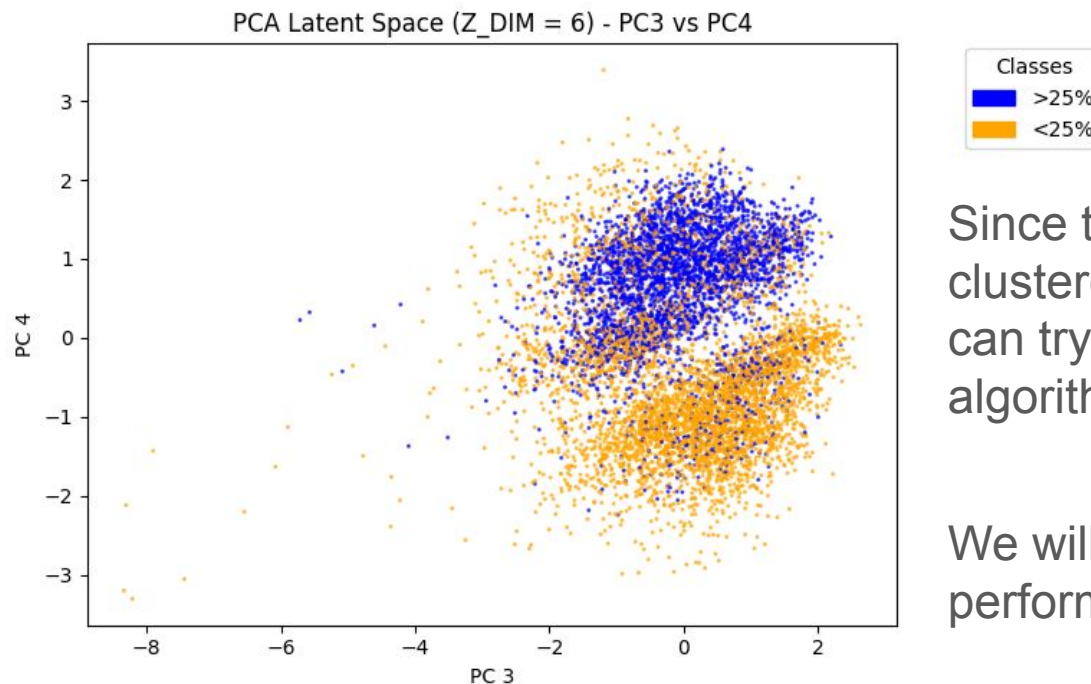




We repeated the process for all zones, obtaining promising results:

	Training set	Validation set	Test set	
ZONE	AVG Accuracy	AVG Accuracy	AVG Accuracy	ROC score
NORD	0.84	0.86	0.75	0.89
CNOR	0.90	0.89	0.77	0.85
CSUD	0.79	0.77	0.76	0.89
SUD	0.81	0.77	0.78	0.88
CALA	0.81	0.71	0.76	0.86
SICI	0.89	0.87	0.86	0.94
SARD	0.89	0.83	0.79	0.87

Average test set accuracy: 0.78



Since the latent space looks well clustered according to classes labels, we can try to implement a simple clustering algorithm. In other words:

is NN an overkill?

We will train a SVM and compare the performances

SVMs are **computationally simpler** and **easier to train** than NNs

If the performances are similar, we can tighten the model to a “less expensive” one



SVMs support a wide variety of **kernels**: we tried out many

```
param_grid = {  
    'nu': [0.01, 0.05, 0.1, 0.3],  
    'kernel': ['linear', 'rbf', 'poly', 'sigmoid'],  
    'degree': [2, 3, 4],  
    'gamma': ['scale', 'auto']  
}
```

As usual we performed a **grid search** on hyperparameters space

```
Best Params: {'nu': 0.3, 'kernel': 'rbf', 'gamma': 'scale', 'degree': 2}  
Validation Accuracy: 0.7239366138448707  
Test Accuracy: 0.7175
```

Note: degree is only relevant if the kernel is 'poly'



We trained on all zones and compared results:

Legend
SVM is better
Comparable
SVM is worse

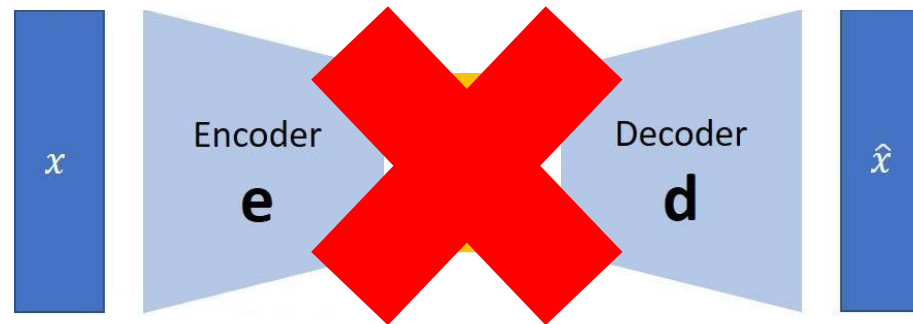
	Training set		Validation set		Test set	
ZONE	NN Accuracy	SVM Accuracy	NN Accuracy	SVM Accuracy	NN Accuracy	SVM Accuracy
NORD	0.84	0.84	0.86	0.88	0.75	0.71
CNOR	0.90	0.64	0.89	0.87	0.77	0.80
CSUD	0.79	0.77	0.77	0.70	0.76	0.51
SUD	0.81	0.43	0.77	0.64	0.78	0.50
CALA	0.81	0.71	0.71	0.62	0.76	0.52
SICI	0.89	0.89	0.87	0.62	0.86	0.68
SARD	0.89	0.88	0.83	0.78	0.79	0.82

SVM performs **generally worse**:

- installation data may be hard to cluster
- clusters may be sensible to different weather conditions

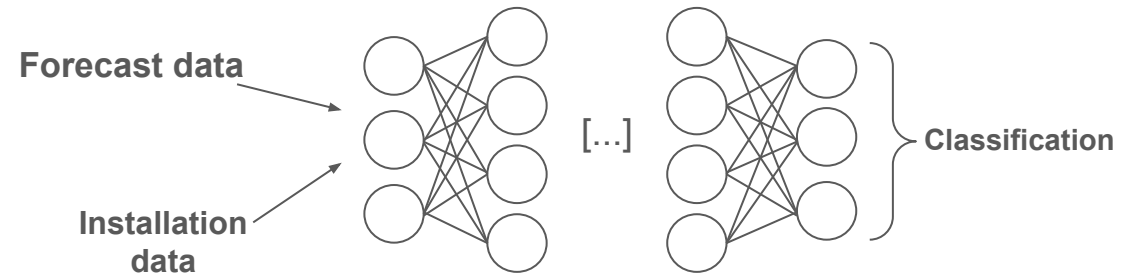
We need a more complicated model

Why don't we just use a NN?



We can **simplify the model** by completely cutting the VAE

The NN has already shown **potential** in handling both installation and meteo forecast





We train the NN **directly** on the **forecast data** (90 features) + fitted **installation data** (3 features)

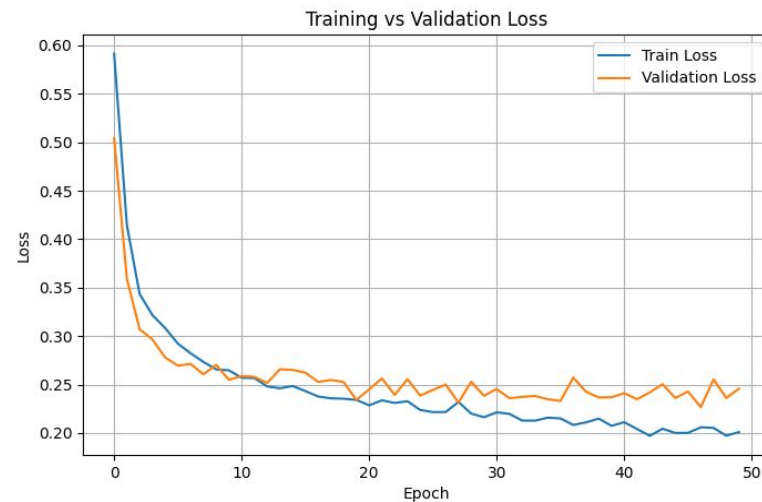
We used the same “settings” (i.e. the parameters for the grid search, number of epochs and division in training-val-test set) to directly compare the two methods (with/out VAE).

```
# Define the search space
param_grid = {
    'hidden_dims': [[64, 32], [32, 16], [16, 8], [8, 4]],
    'lr': [1e-4, 1e-3],
    'batch_size': [64, 128],
    'weight_decay': [1e-2, 1e-3, 1e-4],
    'dropout_prob': [0.05, 0.15, 0.3]
}
```

Best Hyperparameters:

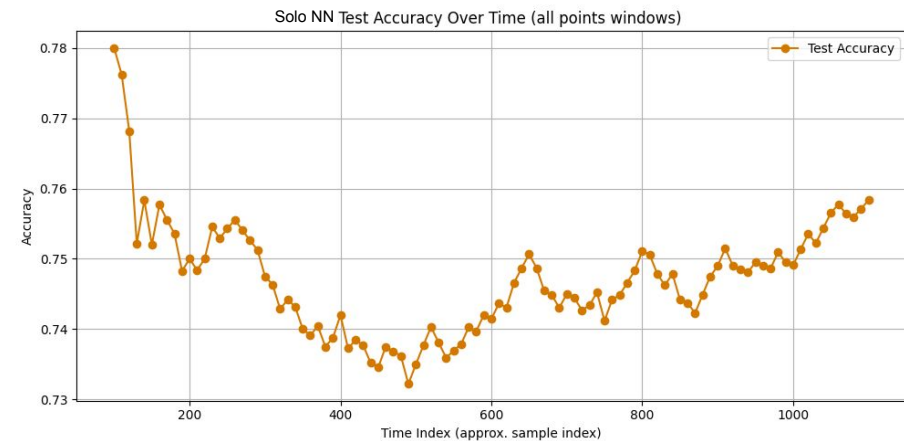
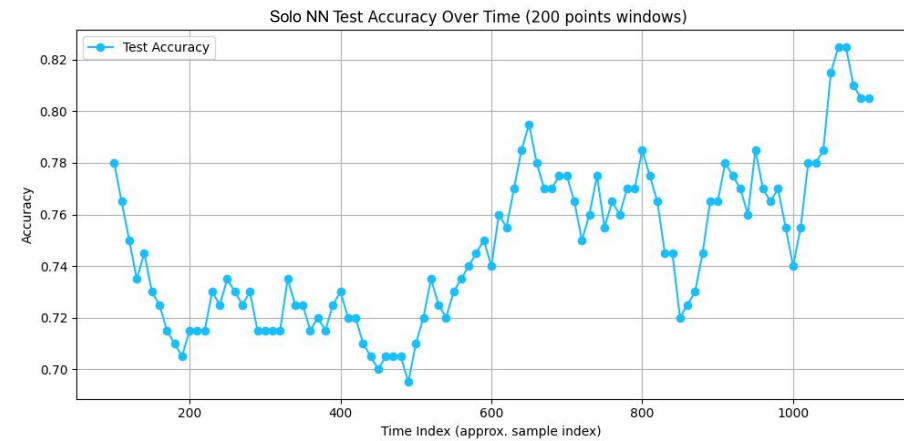
```
{'hidden_dims': [32, 16], 'lr': 0.001, 'batch_size': 128, 'weight_decay': 0.0001, 'dropout_prob': 0.15}
```

Best Validation Accuracy: 90.25%



Test accuracy over time follows a weird trend: NN is not really suitable for time series

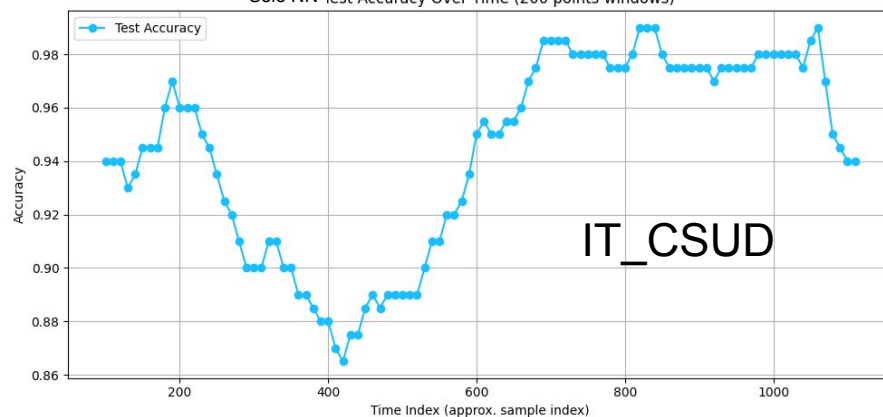
It interprets well **local trends**, but not long term patterns



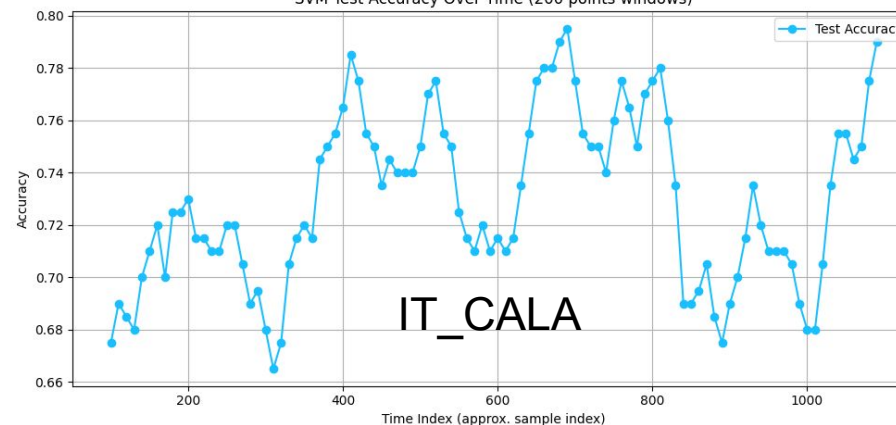


NN solo results (other zones)

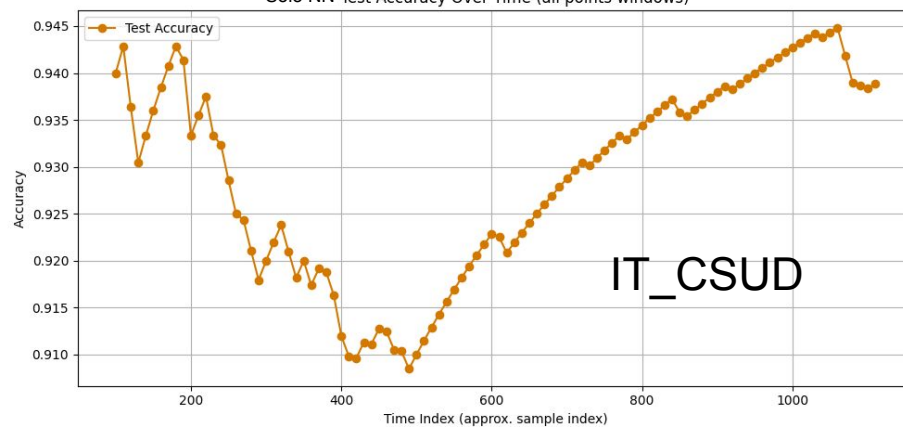
Solo NN Test Accuracy Over Time (200 points windows)



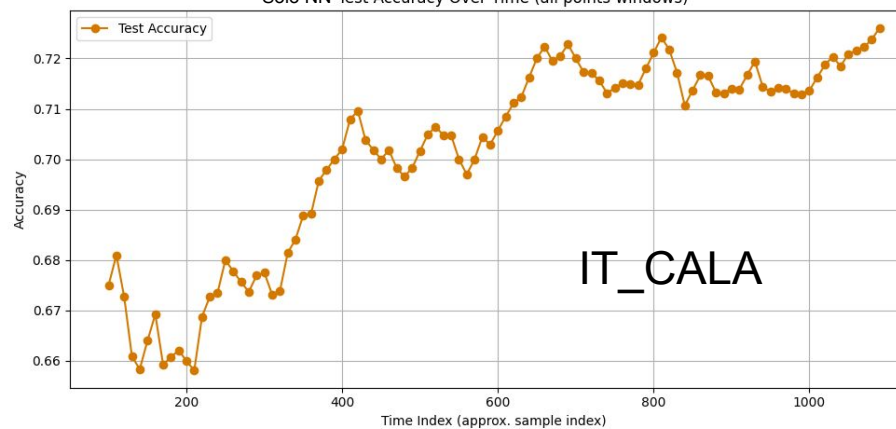
Solo NN SVM Test Accuracy Over Time (200 points windows)



Solo NN Test Accuracy Over Time (all points windows)



Solo NN Test Accuracy Over Time (all points windows)





We trained on all zones and compared results:

Legend
solo NN is better
Comparable
solo NN is worse

	Training set		Validation set		Test set	
ZONE	VAE + NN Accuracy	solo NN Accuracy	VAE + NN Accuracy	solo NN Accuracy	VAE + NN Accuracy	solo NN Accuracy
NORD	0.84	0.91	0.86	0.78	0.75	0.76
CNOR	0.90	0.96	0.89	0.91	0.77	0.92
CSUD	0.79	0.96	0.77	0.84	0.76	0.70
SUD	0.81	0.88	0.77	0.84	0.78	0.85
CALA	0.81	0.86	0.71	0.66	0.76	0.81
SICI	0.89	0.93	0.87	0.87	0.86	0.83
SARD	0.89	0.92	0.83	0.90	0.79	0.89

Solo NN performs **quite well**: performance is not consistent, but **nice** overall

Problem lies in **time-aware** accuracy: NN learns better local time trends than long time patterns.

VAE + Regression: energy production prediction



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

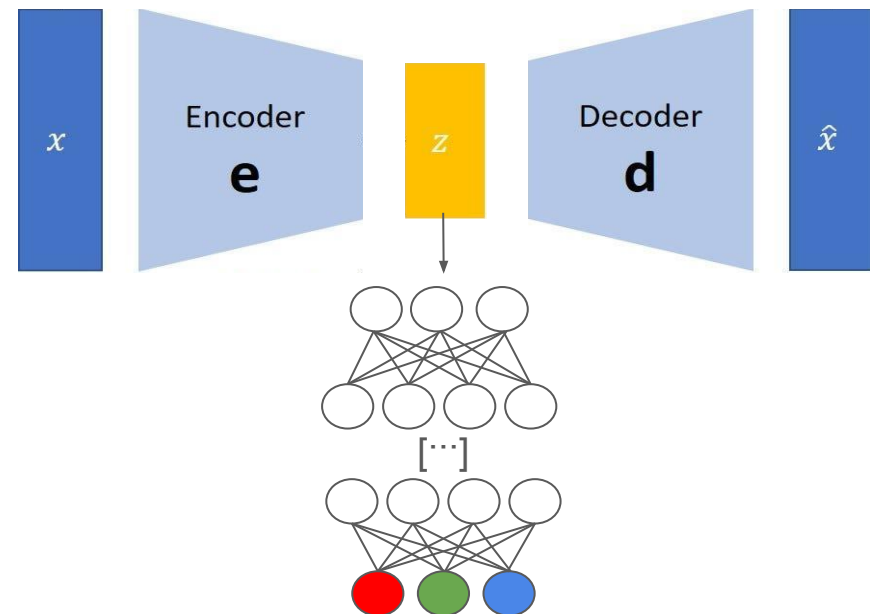


VAE

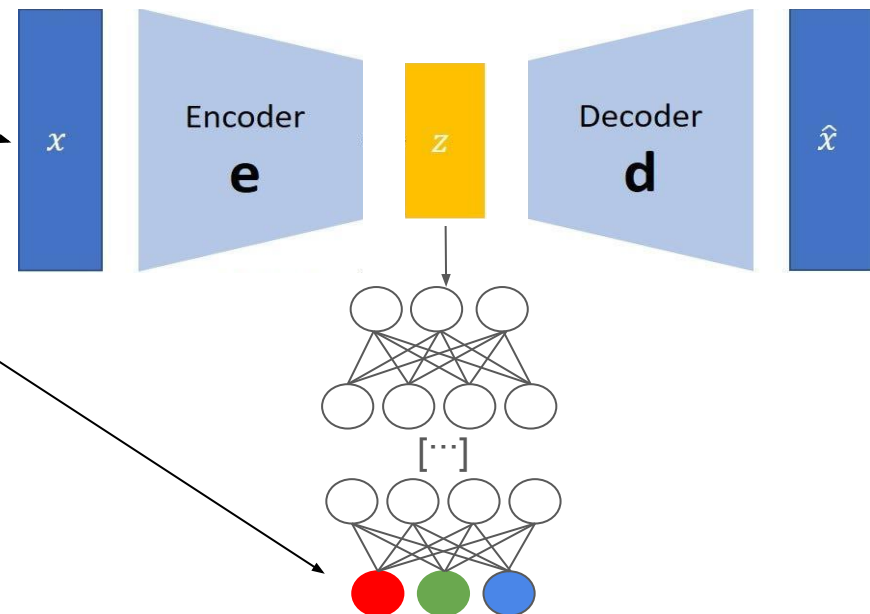
- **2 hidden layers** for the encoder and **1 hidden layer** for the decoder

NN

- **2 hidden layers** with **dropout** ($p=0.1$) and **Batch Normalization** layers
- **ReLU** activation function in the **output layer**, since all labels are positive valued (normalized energy production)



- **VAE Inputs** : meteorological and temporal data every 3 hours in sliding windows of 24 hours
- **Labels**: we considered different options. In our first trials, **3-dim labels** with **solar**, **wind** and **hydro-electric** energy production normalized to the installed power, every 3h



Here we feed the NN exclusively with the latent space vectors **z** and **train** the whole **VAE+NN** architecture at the **same time**,

defining an overall loss: $L_{tot} = \beta L_{VAE} + (1 - \beta) L_{Regr}$ ($0 < \beta < 1$)



With 3-dim vectors as labels, our predictions for **hydro-electric** production were consistently significantly **worse**.

We found the **hydro-electric** energy production is in fact the **least correlated** with weather-related variables.

Label	Mean absolute correlation with input features
Wind	0.231
Solar	0.216
Hydro-electric	0.179

Including weakly correlated label vector component can negatively influence learnt weights for the other components as well.



We decide to define 1-dim label vectors, considering **single predictions** and repeating training separately **for each energy source**



First, we look for a **preliminary value of β** allowing to obtain stable learning and an almost equal **balance** between **regression** and **VAE loss**. We initially fix this value to perform a first **grid search**, with objective the minimization of the **Regression Validation loss**. This is the initial hyper-parameters space:

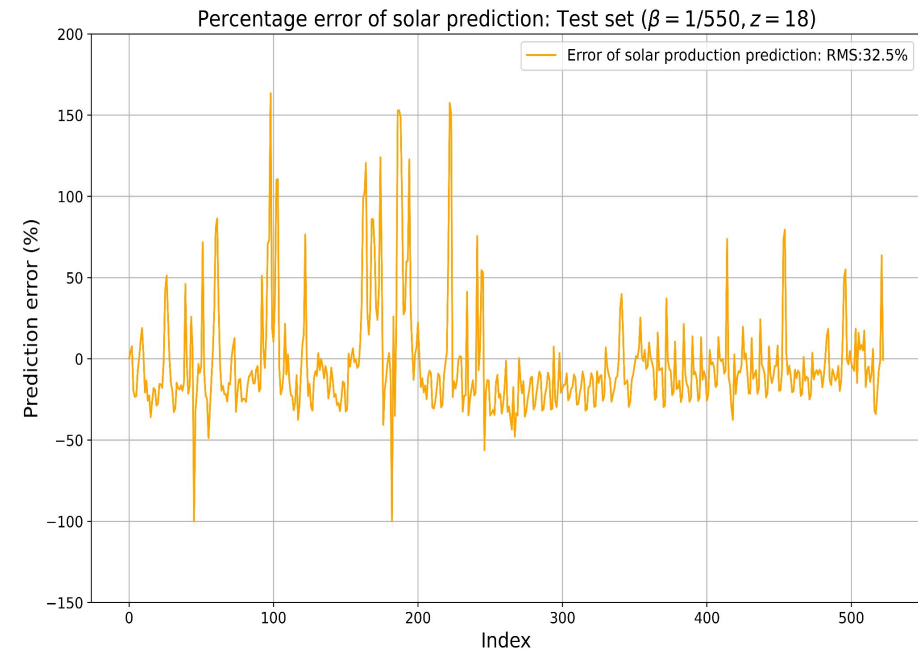
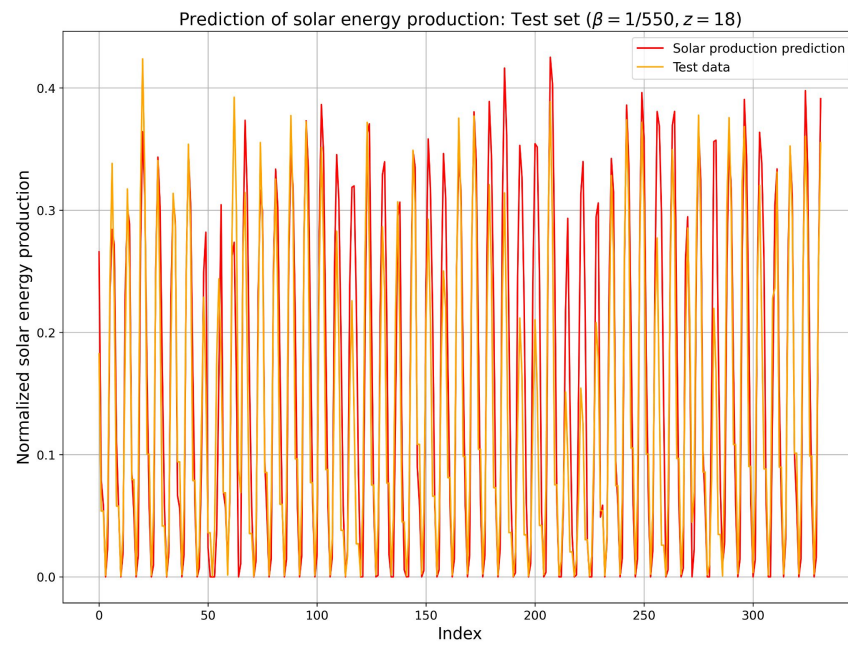
```
# VAE architecture
hidden_dim1 = trial.suggest_categorical("hidden_dim1", [64,128,256])
hidden_dim2 = trial.suggest_categorical("hidden_dim2", [64,128,256])
latent_dim = trial.suggest_categorical("latent_dim", [6,8,10,12,14,16,18,20])
hidden_dim3 = trial.suggest_categorical("hidden_dim3", [32,64,128,256])

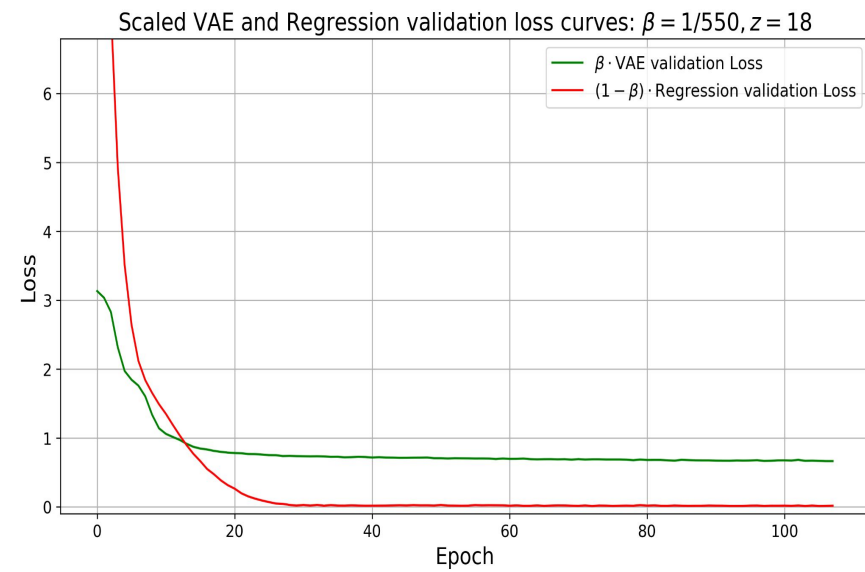
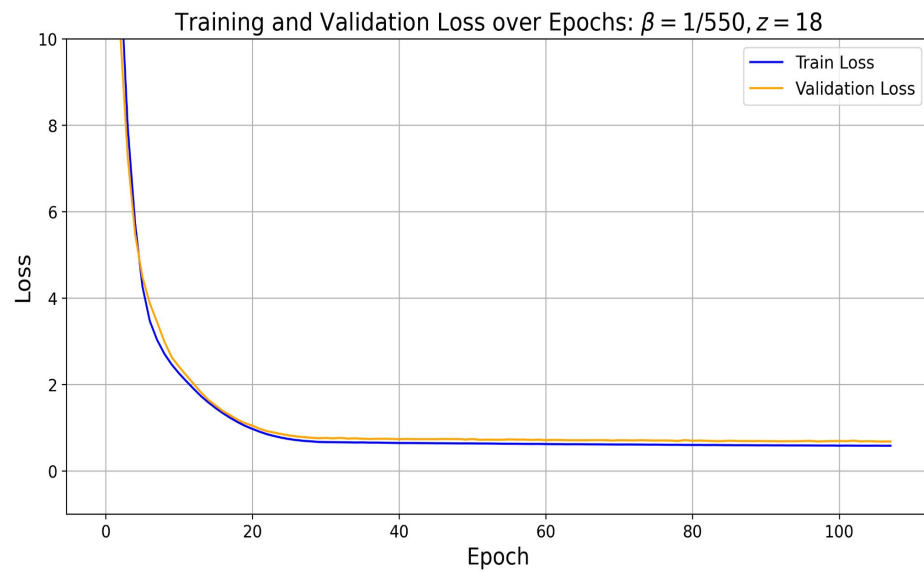
# Regression architecture
N1 = trial.suggest_categorical("N1", [16,32,64,128,256])
N2 = trial.suggest_categorical("N2", [16,32,64,128,256])

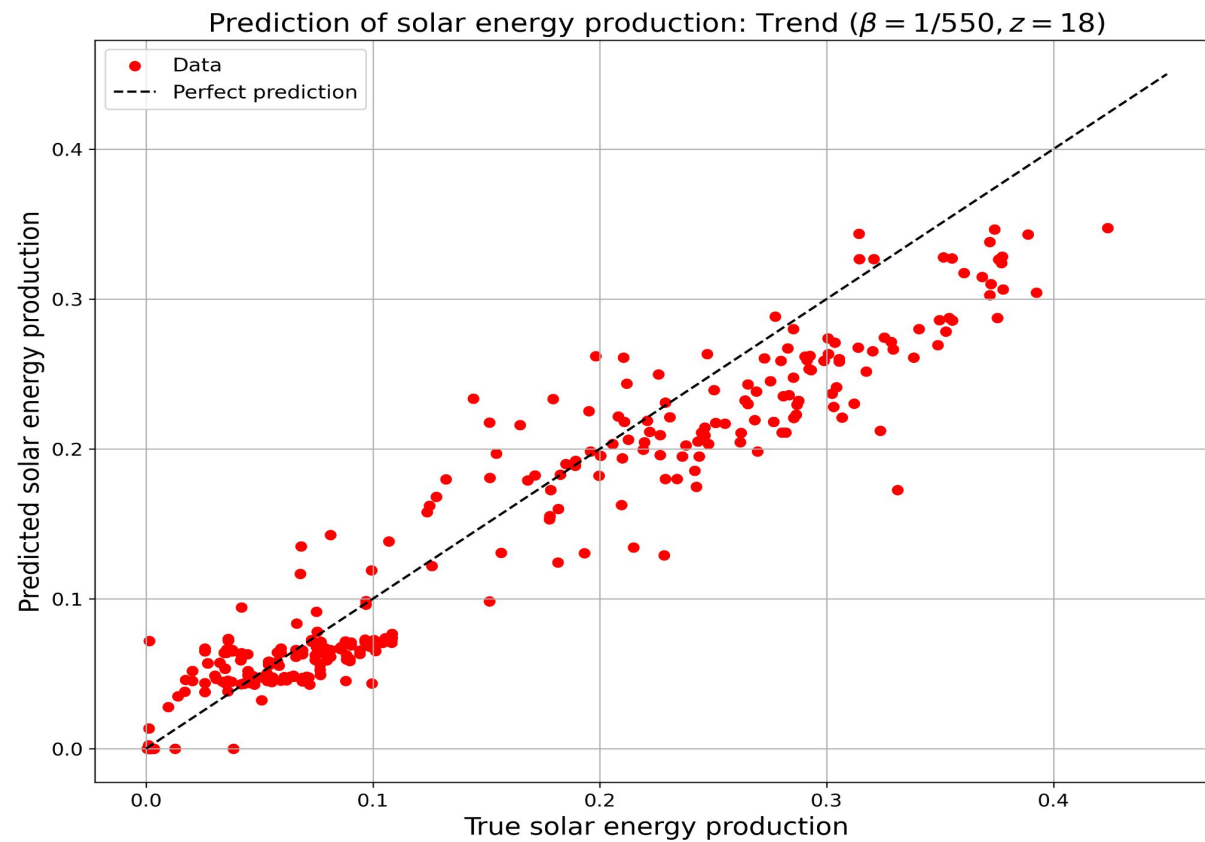
lambda_tik = trial.suggest_float("lambda_tik", 5.0*1e-6,1e-4, log=True)
lr = trial.suggest_float("lr",1e-5,1e-4, log=True)
act_name = trial.suggest_categorical("activation", ["relu","sigmoid","leaky_relu"])
```



- We then **finely tune** the value of β in the definition of the loss to obtain the lowest possible final Regression Validation loss: consider a search in a reasonable range for $1/\beta$
- We implement another grid search to tune the **learning rate** and the L^2 regularization parameter λ , fixing every other parameter.
- **Retrain** the architecture **on the whole training set** with the best hyper-parameters found
- **Predictions:** consider the **average** over 10^4 passes to account for variability in the latent space distribution
- Repeat the same process individually for all energy sources and all the market areas



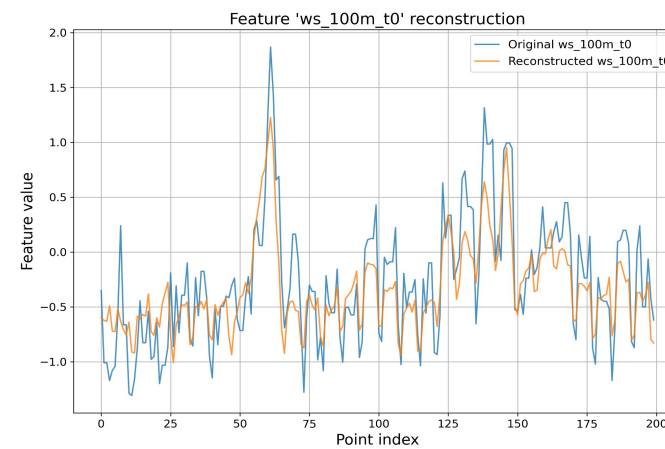
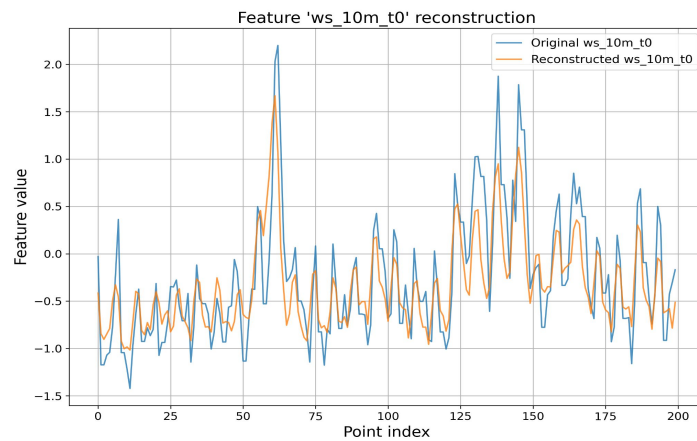
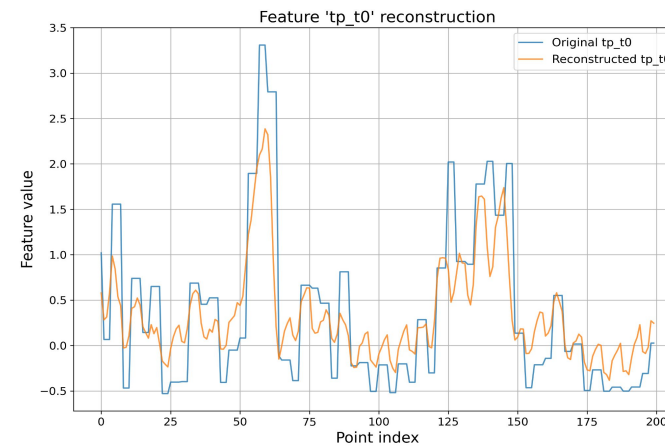
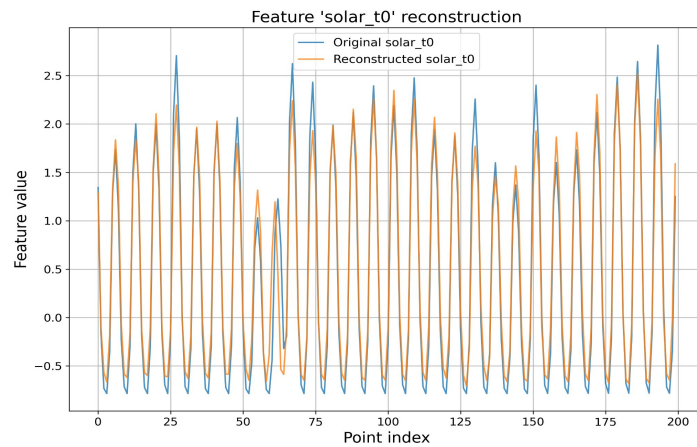


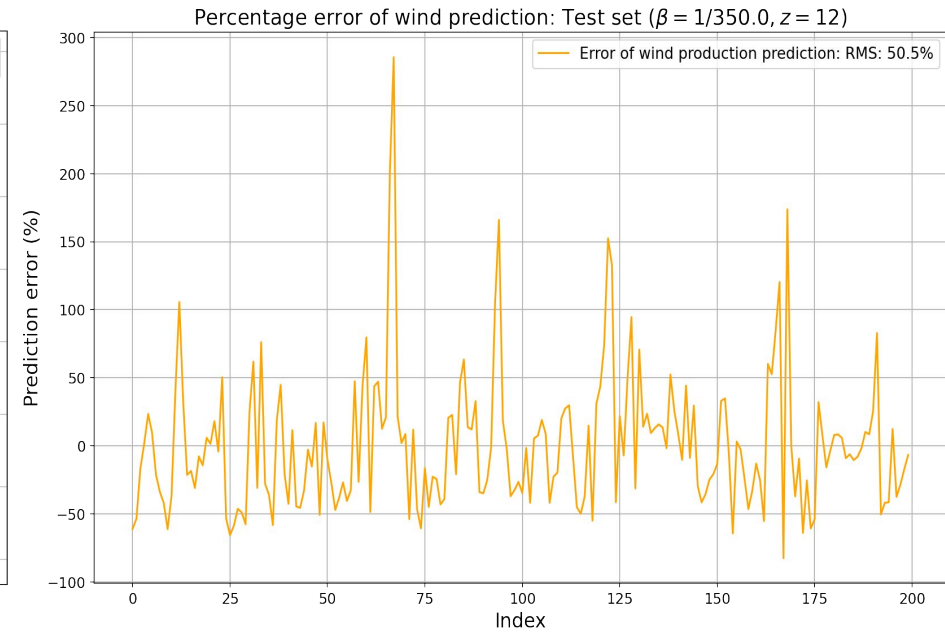
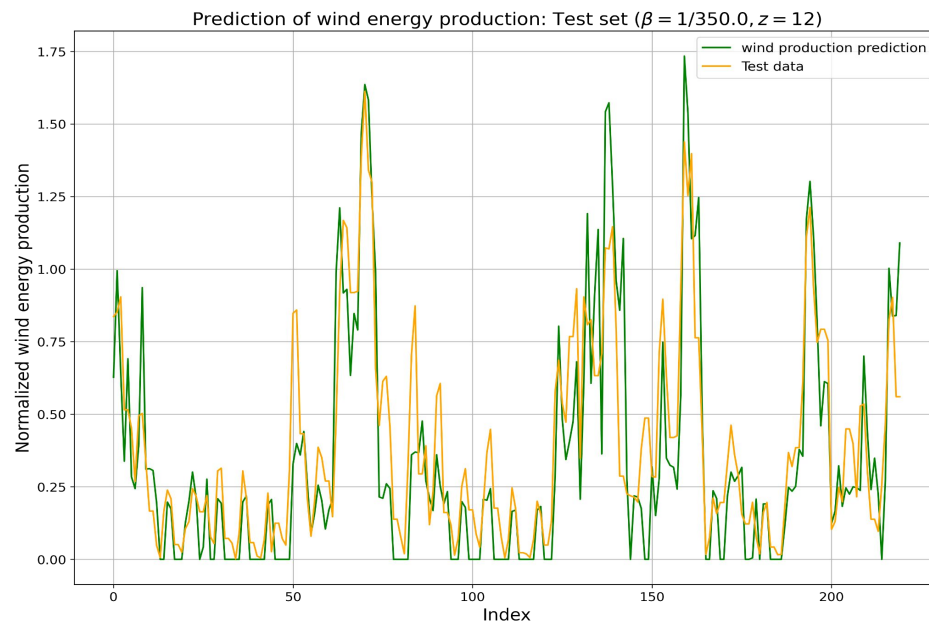




UNIVERSITÀ
DEGLI STUDI
DI PADOVA

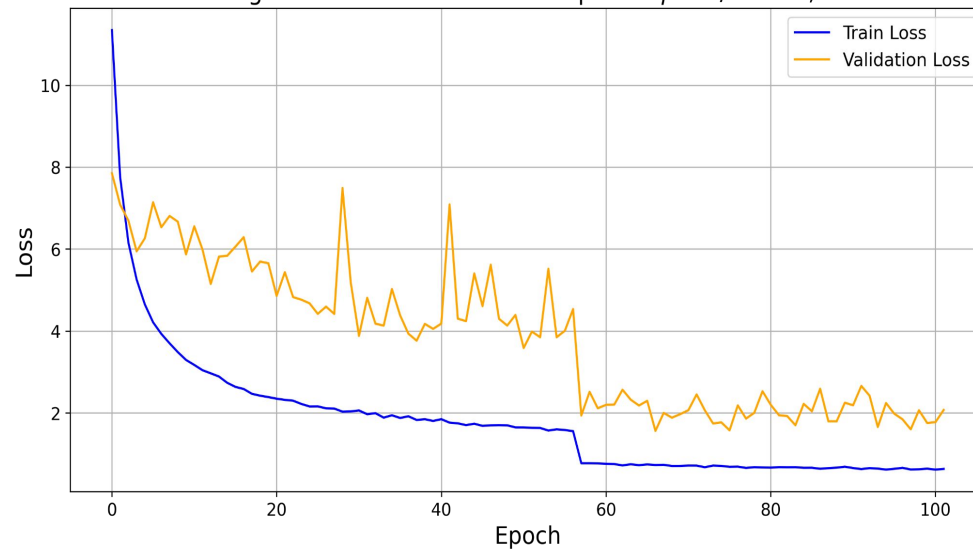
Features reconstruction (NORD-Solar) 58



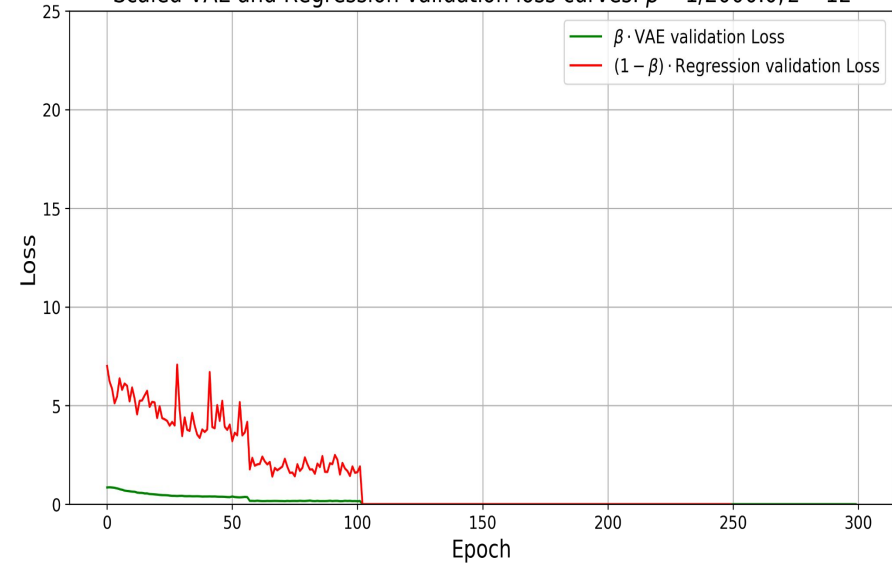


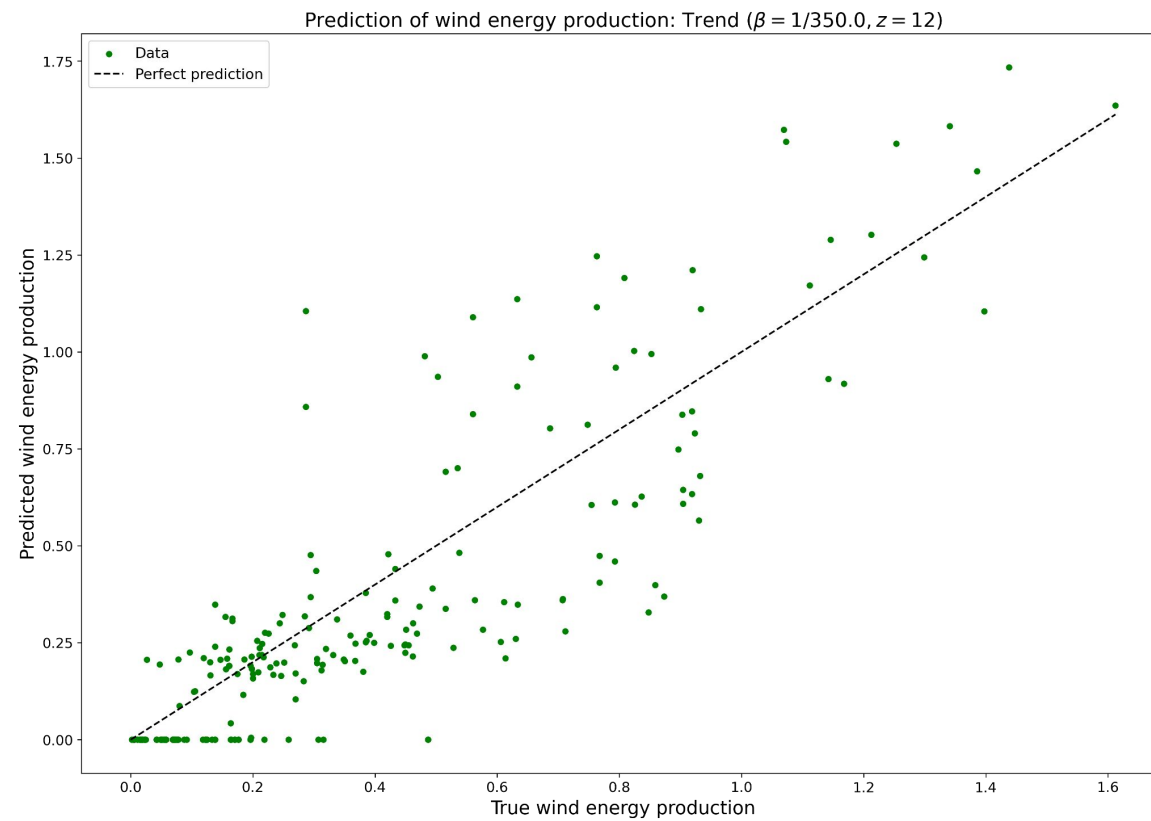


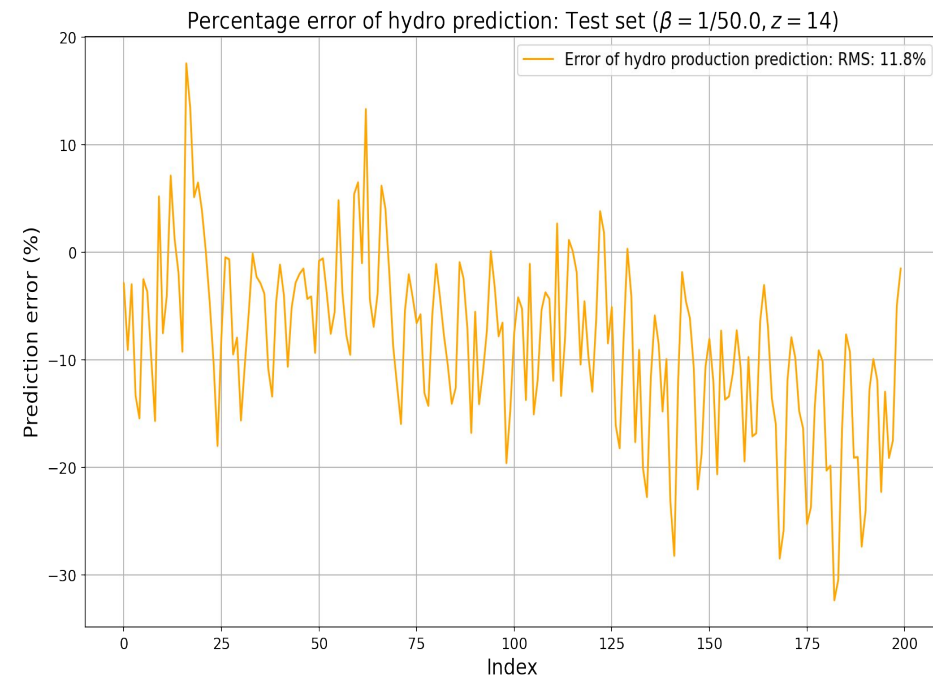
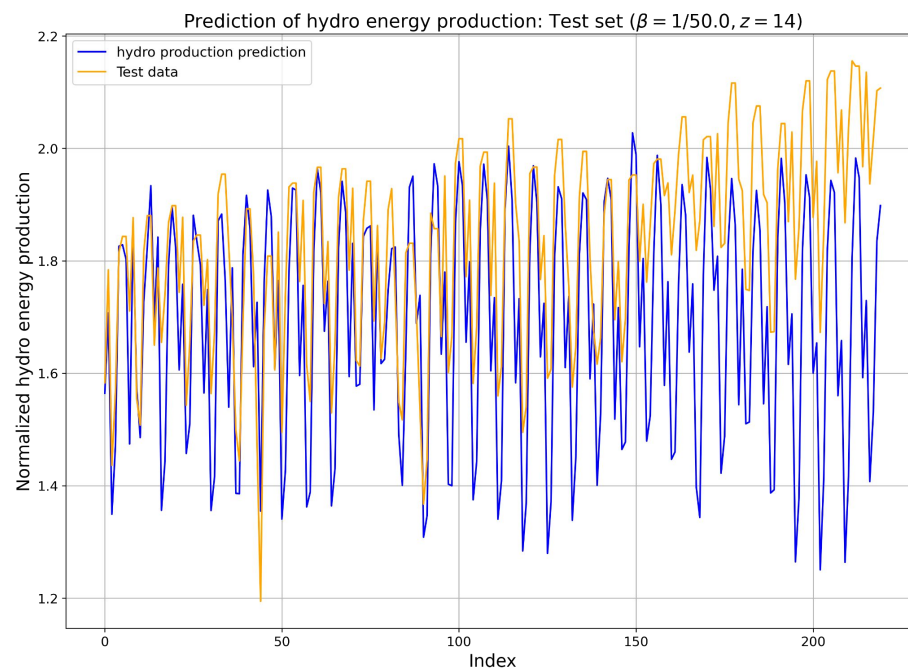
Training and Validation Loss over Epochs: $\beta = 1/2000.0, z = 12$



Scaled VAE and Regression validation loss curves: $\beta = 1/2000.0, z = 12$

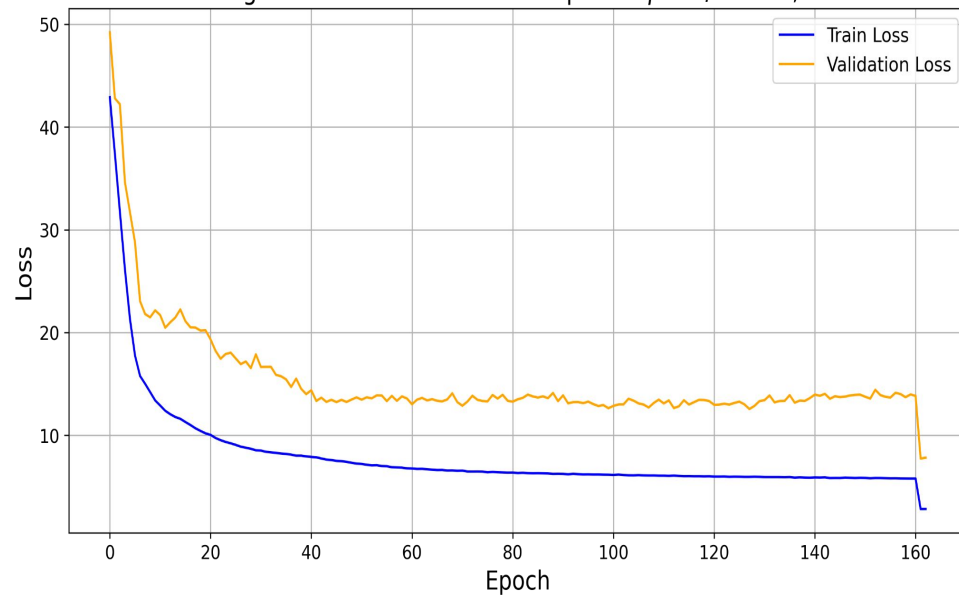




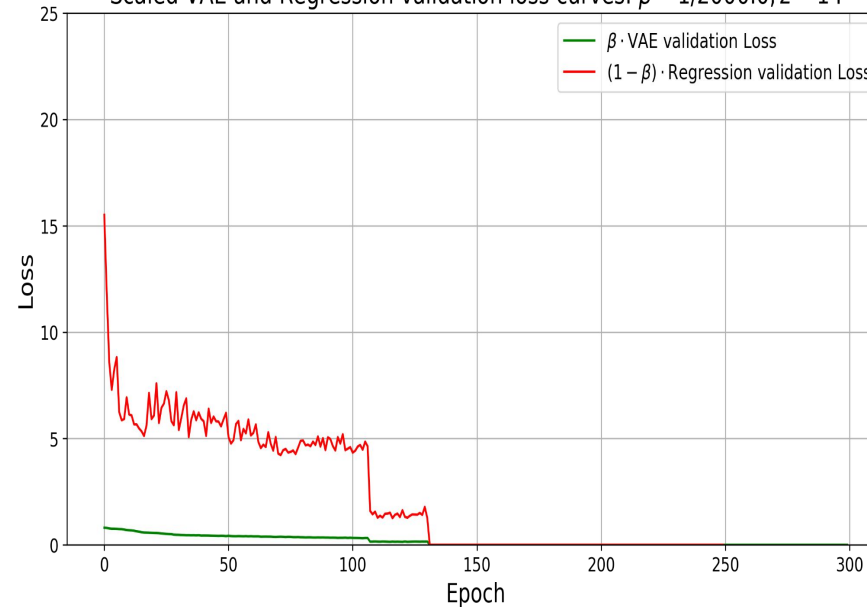




Training and Validation Loss over Epochs: $\beta = 1/2000.0, z = 14$

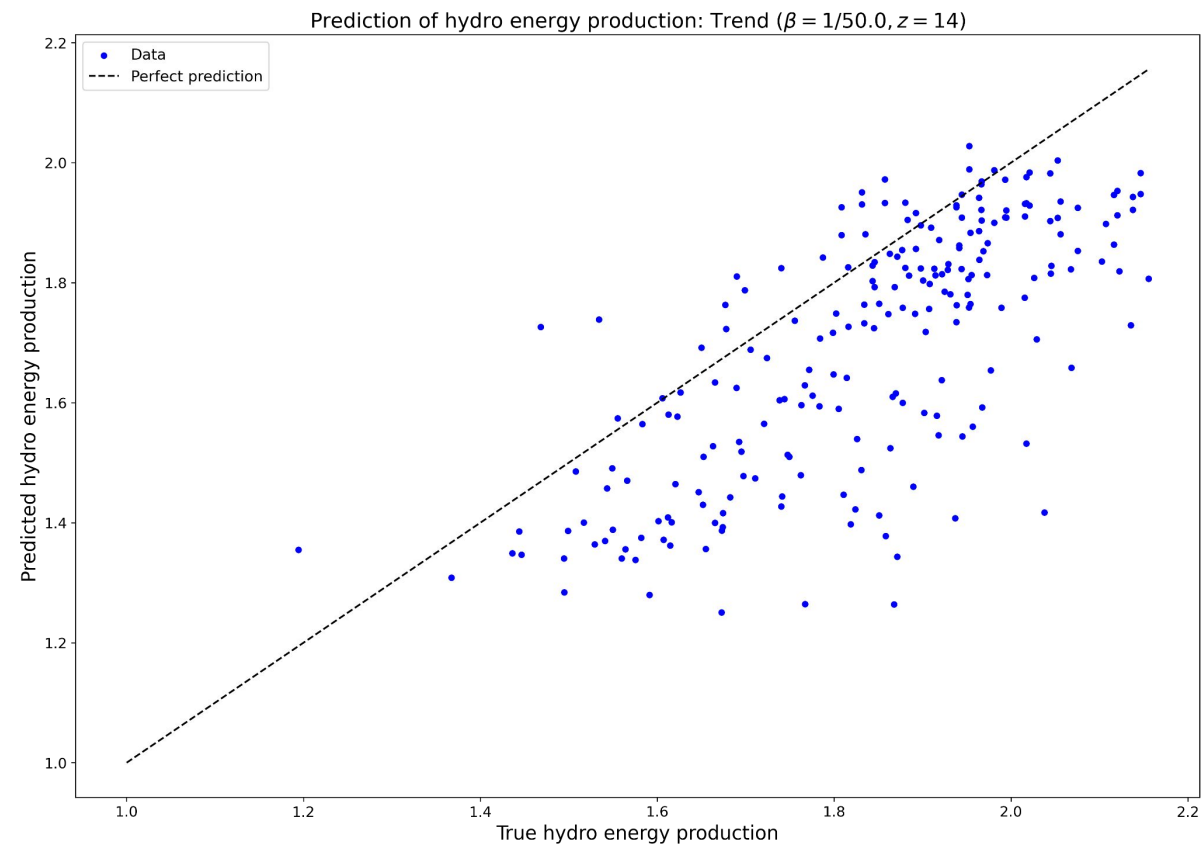


Scaled VAE and Regression validation loss curves: $\beta = 1/2000.0, z = 14$





Results: NORD-Hydro





- We compute the **percentual RMS error** on the test set and compare our results with **reference values** from the best predictive methods available in 2024 (in **green** cases in which **VAE+NN overperforms**)

ZONE	SOLAR RMS	WIND RMS	HYDRO RMS
NORD	32.5%	132.6%	11.8%
SUD	64.6%	141.4%	27.2%
SICI	65.5%	275.5%	74.3%
SARD		50.5%	

- The best results are clearly obtained for the solar and hydro features, while our model for **wind generally strongly underperforms** reference values.
- We managed to **improve** predictions for the **first** (Hydro-Nord) and **third** (Solar-Nord) **most energy-producing sources** in the market in 2024 **by 16% and 42% respectively**

Regression: NN without VAE

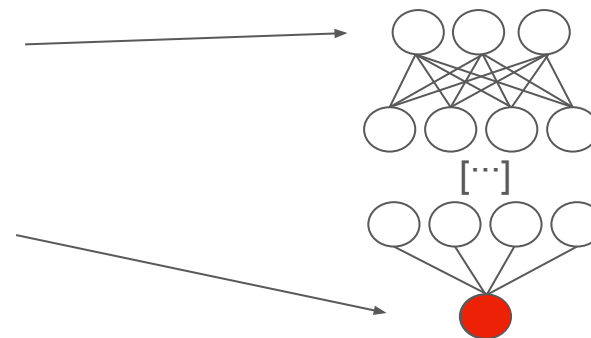
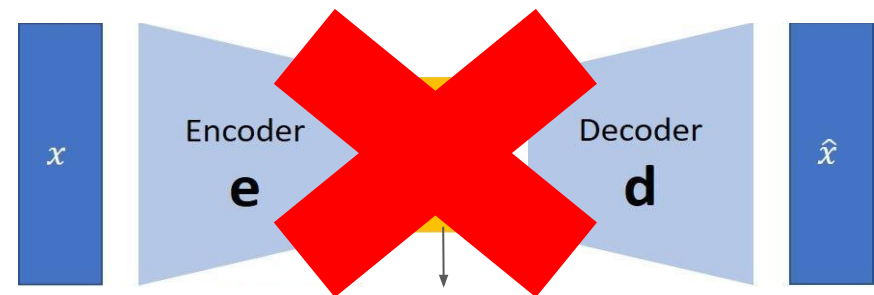


UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Again, is the use of VAE justified?

We trained simple FFNNs to verify this!

- **Inputs** : meteorological and temporal data every 3 hours in sliding windows of 24 hours
- **Labels**: 1-dim labels with **solar**, **wind** and **hydro-electric** energy production normalized to the installed power, every 3h





We train the NN **directly** on the **forecast data** (90 features) and **1 target** at time (solar - hydro - wind).

Also here we used the same architecture of the NN used in the first part (with the VAE) to directly compare the two methods.

We implemented a Grid Search on the following hyperparameters:

```
# Sample hyperparams
hidden_dims = trial.suggest_categorical("hidden_dims", [(128, 64), (64, 32), (32, 16), (16, 8)])
dropout = trial.suggest_float("dropout", 0.1, 0.5, step=0.1)
lambda_tik = trial.suggest_float("lambda_tik", 1e-4, 1e2, log=True)
lr = trial.suggest_categorical("lr", [1e-5, 1e-4, 1e-3])
batch_size = trial.suggest_categorical("batch_size", [8, 16, 32])
act_fun = trial.suggest_categorical("act_fun", ['relu', 'leaky_relu', 'sigmoid'])
```



ZONE	SOLAR RMS		WIND RMS		HYDRO RMS	
	VAE+NN	NN	VAE+NN	NN	VAE+NN	NN
NORD	32.5%	37.6%	132.6%	851.5%	11.8%	24.2%
SUD	64.6%	35.1%	141.4%	336.7%	27.2%	19.5%
SICI	65.5%	51.9%	275.5%	186.5%	74.3%	107.1%
SARD			50.5%	222.8%		

- NN appears to perform similarly or better to predict solar production, while for the other features, especially wind, it appears VAE+NN is generally preferable
- Anyhow, unlike VAE+NN, in none of the cases NN alone managed to improve results compared to our reference values for 2024

Final remarks



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Classification

Implement **multiclass classification** problem to increase the information on the renewable energy production

Regression

A more powerful model (e.g. **LSTMs, Transformers**) appears to be necessary, especially to improve wind predictions

More finely grained precipitations and wind **data** are likely necessary to improve predictive power



Use of load data as feature

Power plants are useful to generate more electricity in time spans when the demand is high.



There should be a relevant correlation between general energy consumption (load) and production: this is mostly true for **hydro-electric power** (we can directly control the production). We could use load forecasts or predictions as another input feature

More in general, since we implemented our model to predict single labels, we can introduce other specific features based on the energy source we are interested in